前端高频八股-补充版

1.清除浮动

设置BFC即可清楚浮动

```
.container {
    display: flow-root; /* 这一行代码就能完美解决高度塌陷，无需伪元素 */
}
```

```
.clearfix::after {
    content: "";          /* 1. 必须有 content 属性，否则伪元素不显示 */
    display: table;       /* 2. 将伪元素设为块级或表格布局 */
    clear: both;          /* 3. 核心：清除左右两侧的浮动影响 */
}

/* 如果需要更好的兼容性（针对老旧浏览器） */
.clearfix {
    *zoom: 1;             /* 触发 IE6/7 的 HasLayout ，已废弃*/
}
```

2.行内元素和块级元素如何设置居中

Flex 布局（通杀所有元素）

这是目前实际开发中最推荐的方法，**不需要区分**内部是行内还是块级。

只需在**父元素**上写三行代码：

CSS

```
.parent {
    display: flex;
    justify-content: center; /* 水平居中 */
    align-items: center;     /* 垂直居中 */
    height: 300px;           /* 必须有高度才能看到垂直居中 */
}
```

3.margin合并

Margin 合并（Margin Collapsing），也叫外边距重叠，是 CSS 中一个非常“反直觉”的特性。简单来说，就是当两个垂直方向的外边距相遇时，它们不会相加，而是会**合并成一个外边距**。

合并后的结果通常是：**取两者中的最大值**。

---

1. **Margin 合并的三种常见场景**

A. **相邻兄弟元素合并**

这是最常见的。两个上下排列的 `div`，上面的设置 `margin-bottom: 30px`，下面的设置 `margin-top: 20px`。

- **直觉结果：** 间距 50px。
- **实际结果：** 间距 30px（取大值）。

B. 父子元素合并（塌陷）

如果父元素没有边框（border）或内边距（padding），子元素的 `margin-top` 会直接“传导”给父元素。

- **表现：** 你想让子元素相对于父元素往下挪 20px，结果子元素没动，父元素整体带着子元素一起往下挪了。

C. 空元素自身合并

如果一个空元素没有高度、边框和内边距，它的 `margin-top` 和 `margin-bottom` 会碰到一起并合并。

---

2. **关键规则**
3. **只发生在垂直方向**：水平方向（左右外边距）永远不会发生合并。
4. **取大值**：如果一个是 20px，一个是 30px，合并后是 30px。
5. **正负相加**：如果一个是 30px，一个是 -10px，合并后是 20px。
6. **全负取绝对值最大的**：如果一个是 -20px，一个是 -30px，合并后是 -30px。

---

3. **如何防止 Margin 合并?**

在实际开发中，父子合并（塌陷）往往是我们想要避免的。以下是几种常用的解决方法：

方法一：给父元素加“阻挡物”

只要父子边距不直接接触，就不会合并。

- 给父元素设置 `border`（哪怕是透明的）。
- 给父元素设置 `padding`（哪怕只有 0.1px）。

方法二：触发 BFC（最推荐）

BFC（块级格式化上下文） 是一块独立的渲染区域，它规定 BFC 内部的元素不会与外部合并。

- 给父元素设置 `overflow: hidden;`。
- 给父元素设置 `display: flex;` 或 `display: inline-block;`。

方法三：改变定位或浮动

- 设置 `float: left/right;`。
- 设置 `position: absolute/fixed;`。

4.css定位

在 CSS 中，`position` 属性是控制元素“脱离常规文档流”并进行精准定位的核心工具。

我们可以将定位分为 **5 种模式**。为了方便记忆，你可以把文档流想象成排队，而定位就是各种“插队”或“离队”的规则。

---

1. static (静态定位)

这是所有元素的**默认值**。

- **规则**：遵循正常的文档流，从上到下、从左到右排队。
- **特点**：设置 `top`，`bottom`，`left`，`right` 和 `z-index` 均**无效**。

---

2. relative (相对定位)

元素相对于它**原本在文档流中的位置**进行偏移。

- **规则**：通过 `top` 等属性移动，但它**原本占据的空间依然保留**（不会被后面的元素顶替）。
- **核心用途**：
  1. 做微调。
  2. **最重要的用途**：作为绝对定位（absolute）子元素的**参照坐标系**（即“子绝父相”）。

---

3. absolute (绝对定位)

元素脱离文档流，相对于**最近的非 static 祖先元素**进行定位。

- **规则**：
  - 如果父级、爷爷级...都没有设置定位，则相对于 `<html>` **（视口）** 定位。
  - 如果找到了设置过 `relative/absolute/fixed` 的祖先，就以该祖先的左上角为基准。
- **特点**：**不占用空间**，会漂浮在其他元素上方。

---

4. fixed (固定定位)

元素脱离文档流，永远相对于**浏览器窗口（Viewport）**定位。但是当**元素祖先的 transform 属性非 none 时，会相对于该祖先进行定位。**

- **规则**：无论页面如何滚动，元素的位置都不变。
- **典型场景**：网页顶部的固定导航栏、右下角的“回到顶部”按钮、弹窗遮罩层。

---

5. **sticky (粘性定位)**

它是 `relative` 和 `fixed` 的“混血儿”，非常智能。

- **规则**：在跨越特定阈值（如 `top: 0`）前，它表现得像 `relative`；当页面滚动到该位置时，它会“粘”在屏幕指定位置，表现得像 `fixed`。
- **注意**：必须配合 `top/bottom/left/right` 之一使用才能生效。
- **典型场景**：表格的表头固定、文章列表的字母索引头。

模式	是否脱离文档流	相对于谁定位	是否占据原位
`static`	否	正常流	是
`relative`	否（仅视觉偏移)(子绝父相)	**自身原始位置**	**是**
`absolute`	**是**	**最近的非 static 祖先**	否
`fixed`	**是**	**浏览器窗口**	否
`sticky`	否（直到触发阈值）	离它最近的**滚动祖先**	是

> ## 关键知识点：z-index (层级)
>
> 当多个定位元素重叠时，谁在上面？
>
> - 默认情况下，**后写的元素**在上面。

> - 使用 `z-index: [数字]` 可以手动调整。数字越大，越靠前（离你的眼睛越近）。
> - **注意**：`z-index` 仅对设置了定位（非 static）的元素生效。

5.手写实现instance of

```
//手写实现
function myInstance(L, R) { //L代表instanceof左边，R代表右边
    var RP = R.prototype
        var LP = L.__proto__
        while (true) {
            if (LP == null) {
                return false
            }
            if (LP == RP) {
                return true
            }
            LP = LP.__proto__
        }
}
console.log(myInstance({}, Object));
```

6.选择器

CSS 选择器是 CSS 中最基础也最强大的部分，它负责**“寻找”** HTML 页面中的元素，以便给它们穿上样式的“衣服”。

我们可以根据复杂程度和功能，将选择器分为以下几大类：

---

1. 基础选择器 (Basic Selectors)

这些是你每天都会用到的“基本功”。

选择器类型	语法	说明	示例
标签选择器	`div`	选中页面中所有的该标签	`p { color: red; }`
类选择器	`.className`	选中所有带有该 `class` 的元素	`.box { width: 100px; }`
ID 选择器	`#idName`	选中具有该唯一 `id` 的元素	`#header { height: 60px; }`
通配符选择器	`*`	选中页面中的所有元素	`* { margin: 0; }`

---

2. 复合选择器 (Combinators)

通过结合多个基础选择器，精准定位 HTML 结构中的特定元素。

- **后代选择器 (空格)**： `div p` 选中 `div` 内部所有的 `p` 标签（无论嵌套多深）。
- **子代选择器 (`>`)**： `div > p` 只选中 `div` 的**直接子级** `p` 标签。
- **并集选择器 (`,`)**： `h1, h2, p` 同时选中多种元素，共用一套样式。

- **交集选择器：** `p.special` 选中既是 `p` 标签又带有 `.special` 类的元素。
- **相邻兄弟选择器 (`+`)：** `h1 + p` 选中紧跟在 `h1` 后面那**一个** `p` 元素。

---

3. 伪类选择器 (Pseudo-classes)

用于选择元素的**特定状态**（如鼠标悬停、第几个子元素）。

- **状态伪类：**
  - `:hover`：鼠标悬停时。
  - `:active`：鼠标按下未松开时。
  - `:focus`：获得焦点（常用于输入框）。
- **结构伪类：**
  - `:first-child`：选中作为父元素第一个孩子的元素。
  - `:last-child`：选中最后一个孩子。
  - `:nth-child(n)`：最强大的选择器之一，可以选中第 n 个、奇数个（odd）或偶数个（even）元素。

---

4. 属性选择器 (Attribute Selectors)

根据 HTML 属性及其值来选择元素。

- `[title]`：选中带有 `title` 属性的所有元素。
- `input[type="text"]`：选中所有类型为文本的输入框。
- `a[href^="https"]`：选中 `href` 以 "https" **开头**的链接。

---

5. 权重（优先级）计算

当多个选择器指向同一个元素时，浏览器会根据**权重**决定听谁的。

公式计算规则（从高到低）：

1. `!important`：无敌的存在（不建议滥用）。
2. **行内样式** (`style="..."`)：权重值 **1000**。
3. **ID 选择器**：权重值 **100**。
4. **类、伪类、属性选择器**：权重值 **10**。
5. **标签、伪元素选择器**：权重值 **1**。
6. **通配符、继承**：权重值为 **0**。

> **小技巧：** 权重是可以累加的。例如 `.content .title` (10+10=20) 的优先级高于 `.title` (10)。

7.属性继承

凡是影响“文字内容”展示的属性，通常都能继承；而涉及“盒子模型”（排版/布局）的属性，通常不继承

强制继承：`inherit`

```
.parent { border: 1px solid black; }
.child { border: inherit; } /* 子元素也会出现同样的边框 */
```

所有元素可以继承的属性

1、元素可见性：

- visibility

2、光标属性：

- cursor

可继承

- 字体属性
- 文本属性
- 元素可见性
- 列表布局属性
- 光标属性

不可继承

- 盒子模型属性
- display
- 背景属性
- 定位属性
- 生成内容、轮廓样式、页面样式、声音样式

line-height继承方式

- 父元素的 line-height 写了**具体数值**，比如 30px，则子元素 line-height 继承该值。
- 父元素的 line-height 写了**比例**，比如 1.5 或 2，则子元素 line-height 也是继承该比例。
- 父元素的 line-height 写了**百分比**，比如 200%，则子元素 line-height 继承的是父元素 fontSize * 200% 计算出来的值

8.文本溢出

在网页设计中，当内容（通常是文字）超过了容器设定的宽度或高度时，就会发生“溢出”。CSS 提供了几种优雅的方式来处理这种情况，而不是让文字乱跑。

处理文本溢出通常分为**单行文本处理**和**多行文本处理**。

---

1. 单行文本溢出（最常用）

要实现单行文本溢出显示省略号（...），必须同时满足以下 **3 个条件**：

1. `white-space: nowrap;`：强制文字在一行内显示，不允许换行。
2. `overflow: hidden;`：溢出的部分隐藏起来。
3. `text-overflow: ellipsis;`：核心属性，将溢出部分替换为省略号。

CSS

```
.single-line {
  width: 200px;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis; /* 显示省略号 */
}
```

---

2. 多行文本溢出（webkit）

多行溢出处理相对复杂一些。目前最主流的方法是使用 WebKit 内核提供的私有属性（虽然是私有，但主流浏览器兼容性已非常好）。

- **核心属性：**
  - `display: -webkit-box;`：必须结合弹性伸缩盒子模型使用。
  - `-webkit-line-clamp: 3;`：限制显示的行数（例如 3 行）。
  - `-webkit-box-orient: vertical;`：设置伸缩盒子的子元素排列方式。
  - `overflow: hidden;`：隐藏超出部分。

CSS

```
.multi-line {
  width: 200px;
  display: -webkit-box;
  -webkit-line-clamp: 2; /* 限制只显示两行 */
  -webkit-box-orient: vertical;
  overflow: hidden;
}
```

---

3. 核心属性详解

`text-overflow`

这个属性只决定**如何提示**用户内容溢出了。

- `clip`：直接截断，不显示省略号（默认值）。
- `ellipsis`：显示省略号。

`white-space` (空格处理)

- `normal`：默认，自动换行。
- `nowrap`：强制不换行。
- `pre`：保留空格和换行符，行为类似 `<pre>` 标签。

`word-break` & `overflow-wrap` (断句处理)

如果你有一串超长的连续英文单词（比如网址），它们默认不会自动换行。

- `word-break: break-all;`：强行断开单词。
- `overflow-wrap: break-word;`：尽量在单词末尾换行，如果单词太长实在放不下，再从中间断开。

3.多行文本溢出（伪元素+绝对定位）

这种方法的逻辑是：给容器设置固定高度（行高的倍数），然后将一个伪元素（`::after`）绝对定位到容器的**右下角**。如果文字溢出了，伪元素就会覆盖在最后几个字上面，形成省略效果。

```
.multi-line-pseudo {
  position: relative;
  line-height: 1.5em;       /* 行高 */
  max-height: 4.5em;        /* 限制 3 行 (1.5 * 3) */
  overflow: hidden;         /* 隐藏超出部分 */
  width: 300px;
  background-color: white;
}

.multi-line-pseudo::after {
  content: "...";           /* 自定义省略符号 */
  position: absolute;
  bottom: 0;
```

```
  right: 0;
  padding: 0 5px 1px 20px; /* 给左侧留出渐变空间 */

  /* 制作背景渐变，让省略号与文字融合更自然 */
  background: linear-gradient(to right, transparent, white 50%);
}
```

9.CSS3新特性

CSS3（层叠样式表第3版）是 CSS 演进中的一个里程碑。它不再是一个单一的规范，而是拆分成了多个模块，这使得浏览器可以更灵活地支持新功能。

以下是 CSS3 中最重要且最改变开发规则的新特性：

---

1. 布局与盒子模型 (Layout & Box Model)

Flexbox (弹性盒子布局)

彻底解决了居中、对齐和等高布局的难题。通过 `display: flex;`，你可以轻松控制子元素的排列方向、缩放和空间分配。

Grid (网格布局)

最强大的二维布局系统。它可以同时处理行和列，适合构建复杂的响应式网页框架。

Box-sizing (盒子模型控制)

`box-sizing: border-box;` 改变了宽度的计算方式。设置此属性后，`width` 会包含 `padding` 和 `border`，再也不用担心内边距撑破容器了。

---

2. 视觉增强 (Visual Enhancement)

- **圆角 (`border-radius`)**：告别了用切图做圆角的时代。
- **阴影 (`box-shadow` / `text-shadow`)**：为元素和文字添加立体感。
- **渐变 (`linear-gradient` / `radial-gradient`)**：支持线性渐变和径向渐变作为背景图。
- **透明度 (`opacity` & `RGBA`)**：支持设置整个元素或特定颜色的透明度。

---

3. 动画与变换 (Animation & Transform)

这是 CSS3 让网页“动起来”的核心：

- **Transitions (过渡)**：让属性变化平滑进行（例如：颜色在 0.5s 内慢慢变红）。
- **Animations (动画)**：配合 `@keyframes`，可以创建复杂的、循环的路径动画，无需 JavaScript。
- **Transforms (变换)**：支持对元素进行 `rotate`（旋转）、`scale`（缩放）、`translate`（位移）和 `skew`（倾斜），甚至支持 3D 变换。

---

4. 响应式设计 (Responsive Design)

Media Queries (媒体查询)

这是移动优先设计的基石。它允许你根据屏幕宽度、分辨率、横竖屏等条件应用不同的样式。

CSS

```
@media (max-width: 600px) {
  .sidebar { display: none; }
}
```

---

5. 颜色与字体 (Colors & Fonts)

- **Web Fonts (`@font-face`)**：允许开发者加载自定义字体文件（如 .ttf, .woff），不再受限于用户的系统自带字体。
- **新的颜色模式**：
  - **HSL/HSLA**：基于色相、饱和度、亮度。
  - **RGBA**：支持透明通道。

---

6. 其他重要特性

- **CSS 变量 (Custom Properties)**：使用 `--variable-name` 定义全局变量，极大地提高了代码的可维护性。
- **Filter (滤镜)**：如 `blur()`、`grayscale()`，直接在浏览器端处理图片视觉效果。
- **Calc() 函数**：允许在属性值中进行计算，如 `width: calc(100% - 50px);`。

总结表：CSS3 的核心价值

特性分类	解决的核心问题
Flex/Grid	摆脱了 `float` 布局带来的痛苦。
Transitions/Animation	减少了对 JS 动画库和 Flash 的依赖。
Media Queries	让“一套代码适配全设备”成为可能。
Border-radius/Shadow	极大减少了 UI 设计中切图的使用。

10.三栏布局-圣杯双飞翼

圣杯布局（Holy Grail Layout）和双飞翼布局（Double Wing Layout）是前端开发中经典的布局模型，主要用于解决“**两侧定宽、中间自适应**”且“**中间栏优先加载**”的需求。

虽然这两种模型在现代开发中已逐渐被 Flex 或 Grid 取代，但它们对理解 CSS 盒模型、负边距（negative margin）和文档流具有极高的学习价值。

一、 圣杯布局 (Holy Grail Layout)

圣杯布局的核心是通过父元素的内边距（`padding`）为两侧留出空间，然后利用**相对定位**（`position: relative`）将侧边栏“拨”回正确位置。

1. HTML 结构

中间栏必须放在最前面，以确保浏览器优先渲染核心内容。

HTML

```
<div class="container">
  <div class="center">中间自适应内容（优先加载）</div>
  <div class="left">左侧定宽</div>
  <div class="right">右侧定宽</div>
</div>
```

2. 实现步骤

1. **准备空间**：给父容器 `.container` 设置 `padding: 0 200px`（假设两侧宽度均为 200px），为侧边栏留出空白。
2. **全部浮动**：三个子元素全部设置 `float: left`。
3. **宽度设定**：`.center` 设置 `width: 100%`，`.left` 和 `.right` 设置固定宽度 200px。此时左、右两栏会被挤到第二行。
4. **移动左栏**：
   - 设置 `margin-left: -100%`：让左栏上移到第一行的最左侧，但此时它会遮挡中间栏。
   - 设置 `position: relative; left: -200px;`：利用相对定位将它向左移动，正好掉进父容器左侧的 `padding` 区域。
5. **移动右栏**：
   - 设置 `margin-left: -200px`：让右栏上移到第一行的最右侧，同样会遮挡内容。
   - 设置 `position: relative; right: -200px;`：将其移入父容器右侧的 `padding` 区域。

二、 双飞翼布局 (Double Wing Layout)

双飞翼布局是阿里团队对圣杯布局的改进。它取消了相对定位，转而通过在中间栏**内部增加一个包裹层**，利用子层的 `margin` 来避开遮挡。

1. HTML 结构

HTML

```
<div class="container">
  <div class="center">
    <div class="inner">中间自适应内容</div>
  </div>
  <div class="left">左侧定宽</div>
  <div class="right">右侧定宽</div>
</div>
```

2. 实现步骤

1. **全部浮动**：让 `.center`、`.left`、`.right` 三个主块都 `float: left`。
2. **宽度设定**：`.center` 宽度设为 `100%`，左右两栏固定宽度。
3. **内容避让（核心）**：给内部容器 `.inner` 设置 `margin: 0 200px`。这样即使中间栏被侧栏遮挡，里面的内容也不会被遮住。
4. **移动左栏**：设置 `margin-left: -100%`。左栏直接回到行首，刚好覆盖在 `.center` 的空白 margin 上。
5. **移动右栏**：设置 `margin-left: -200px`。右栏回到行尾，覆盖在右侧空白 margin 上。

三. Flex 布局（现代开发首选）

Flex 是一维布局神器，实现三栏布局最为简洁。

- **实现逻辑**：
  - 父容器设置 `display: flex`。
  - 左右两侧设置固定宽度（如 `width: 200px`）。
  - 中间内容设置 `flex: 1`，使其占据剩余全部空间。
- **代码示例**：

  CSS

  ```
  .container { display: flex; }
  .left, .right { width: 200px; flex-shrink: 0; }
  .main { flex: 1; }
  ```

- **特点**：代码极简，天然支持等高布局，内容顺序（DOM顺序）灵活。

四. Grid 布局（二维布局专家）

Grid 是目前最强大的 CSS 布局方案，通过定义“列”即可完成。

- **实现逻辑**：
  - 父容器设置 `display: grid`。
  - 通过 `grid-template-columns` 直接划分为三列。
- **代码示例**：

  CSS

  ```
  .container {
    display: grid;
    grid-template-columns: 200px 1fr 200px; /* 固定两端，中间自适应 */
  }
  ```

- **特点**：逻辑最清晰，完全不需要对子元素设置偏移或浮动。

五. 定位布局 (Absolute Positioning)

这是你项目中 `src/layout/index.vue` 处理整体框架的思想延伸。

- **实现逻辑**：
  - 父容器 `relative` 定位。
  - 左右两栏使用 `absolute` 定位，分别固定在 `left: 0` 和 `right: 0`。
  - 中间栏通过 `margin` 或 `calc` 计算宽度。
- **代码示例**：

  CSS

  ```
  .left { position: absolute; left: 0; width: 200px; }
  .right { position: absolute; right: 0; width: 200px; }
  .main { margin: 0 200px; } /* 或者 width: calc(100% - 400px); left: 200px; */
  ```

- **特点**：三栏在文档流中可以完全独立，互不影响。

11.垂直水平居中

在前端开发中，居中布局是基础且高频的需求。根据你项目中使用的 **Element Plus** 和 **Flex** 布局习惯，我将居中方式分为水平、垂直以及全居中三个维度进行总结：

一、 水平居中 (Horizontal Centering)

1. 行内元素 / 行内块元素

- **方法**：在父级容器上设置 `text-align: center`。
- **场景**：文字、`<span>`、`<img>`、或设置了 `display: inline-block` 的按钮。

2. 块级元素 (Block-level)

- **方法**：给当前元素设置固定宽度，并使用 `margin: 0 auto`。
- **场景**：固定宽度的 `div` 或卡片。

3. Flex 布局 (推荐)

- **方法**：父容器设置 `display: flex; justify-content: center;`。
- **场景**：你项目中 `Home` 页面底部图标的水平居中就是通过此方式实现的。

二、 垂直居中 (Vertical Centering)

1. 单行文本

- **方法**：设置 `line-height` 等于容器的 `height`。
- **场景**：导航栏文字、按钮文字。

2. Flex 布局

- **方法**：父容器设置 `display: flex; align-items: center;`。
- **项目体现**：在你的 `layout/index.vue` 的菜单项或 `Home` 页面的用户信息头部，头像与文字的垂直对齐逻辑均基于此。

3. 表格布局

- **方法**：父容器设置 `display: table-cell; vertical-align: middle;`。

三、 垂直水平全居中 (Perfect Centering)

这是最常用的布局要求，以下是四种主流实现：

1. Flex 布局 (最简、最常用)

CSS

```
.parent {
  display: flex;
  justify-content: center; /* 水平居中 */
  align-items: center;     /* 垂直居中 */
}
```

- **优点**：代码量最少，不需要知道子元素的宽高。

2. Grid 布局 (最硬核)

CSS

```
.parent {
  display: grid;
  place-items: center; /* 一行搞定水平垂直居中 */
}
```

- **优点**：代码极其精简。

3. 绝对定位 + Transform (万能方案)

CSS

```
.child {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
}
```

- **原理**：先让子元素左上角位于父容器中心，再通过 `translate` 向左上方移动自身宽高的 50% 达到中心点。
- **优点**：不需要知道子元素的具体宽高。

4. 绝对定位 + Margin Auto

CSS

```
.child {
  position: absolute;
  top: 0; bottom: 0; left: 0; right: 0;
  margin: auto;
  width: 100px; /* 必须有固定宽高 */
  height: 100px;
}
```

12.绘制1px的线

在屏幕分辨率日益提高的今天（高 PPI/Retina 屏），直接写 `border: 1px solid black;` 在手机上看起来往往会像 2px 甚至更粗。这是因为 CSS 中的 `1px` 是**逻辑像素**，而在高分屏下，一个逻辑像素可能对应 2 个或 3 个**物理像素**。

为了画出“视觉上真实”的细线，以下是几种最主流的解决方案：

---

1. 最推荐：利用 `transform: scale()` (最精细)

这是目前移动端最通用的方案。原理是先画一个 200% 大小的边框，再将其缩放回 50%。

伪元素配合 `position: absolute` 使用，它就像覆盖在元素上方的一张**透明塑料薄膜**。如果你直接在父元素（比如一个按钮）上设置 `transform: scale(0.5)`，那么**按钮里的文字也会跟着缩小一半**，变得模糊不清

CSS

```
.line {
    position: relative;
}
```

```
.line::after {
    content: "";
    position: absolute;
    left: 0;
    top: 0;
    width: 200%;
    height: 200%;
    border: 1px solid #000;
    transform: scale(0.5);
    transform-origin: left top; /* 缩放中心设定在左上角 */
    box-sizing: border-box;
    pointer-events: none; /* 防止遮挡点击事件 */
}
```

- **优点**：能画出真正的 0.5px（在 Retina 屏上），看起来非常锐利，且支持圆角。
- **适用场景**：按钮边框、列表分割线。

---

2. 线性渐变方案 (`linear-gradient`)

利用背景渐变一半有颜色、一半透明来实现。

CSS

```
.line {
    height: 1px;
    width: 100%;
    background-image: linear-gradient(0deg, #000 50%, transparent 50%);
}
```

- **优点**：适合画横线或竖线，不需要伪元素。
- **缺点**：**无法实现圆角边框**，且在某些安卓设备上可能会有锯齿感。

---

3. SVG 矢量方案 (最稳定)

SVG 渲染是独立于像素密度的，可以精确控制线条。

HTML

```
<svg width="100%" height="1px" style="display: block;">
  <line x1="0" y1="0" x2="100%" y2="0" stroke="#000" stroke-width="1" />
</svg>
```

- **优点**：无论屏幕怎么缩放，线条粗细都非常均匀，不会出现模糊。
- **缺点**：需要增加额外的 HTML 标签。

13.响应式布局

响应式布局（Responsive Web Design, RWD）是一种网页设计理念，其核心目标是：**让网页能够自动识别屏幕宽度，并相应地调整排版布局。**

简单来说，就是“一套代码，全端适用”——无论是 30 英寸的 4K 显示器，还是 5 英寸的智能手机，网页都能看起来很舒服。

---

1. 响应式布局的三大核心技术

① 媒体查询 (Media Queries)

这是响应式的“大脑”。它允许我们根据设备的特征（如宽度、分辨率、横竖屏）来应用不同的 CSS 规则。

CSS

```
/* 当屏幕宽度小于 768px（通常是手机）时执行 */
@media (max-width: 768px) {
  .container {
    flex-direction: column; /* 把横向排列改为纵向堆叠 */
  }
}
```

② 流式布局 (Fluid Grids)

不再使用固定的 `px` 单位，而是使用 **百分比 (`%`)**、`vw/vh` 或 `flex` 布局。这样容器的大小会随父级自动拉伸或收缩。

③ 弹性图片 (Flexible Images)

确保图片不会溢出容器。

CSS

```
img {
  max-width: 100%; /* 图片最大不会超过父元素的宽度 */
  height: auto;    /* 保持比例缩放 */
}
```

---

2. 布局演进：从“栅格”到“容器”

在响应式的发展史中，布局方式经历了显著的变化：

- **早期：百分比布局。**计算复杂，容易出现 1px 的对齐偏差。
- **中期：Bootstrap 栅格系统。**将页面划分为 12 列，通过类名（如 `col-md-6`）快速分配空间。
- **现代：Flexbox & Grid。**
  - **Flexbox (弹性盒子)**：擅长**一维**布局（行或列），处理对齐和分配剩余空间非常优雅。
  - **Grid (网格布局)**：擅长**二维**布局（行和列同时控制），是目前最强大的响应式布局工具。

---

3. 响应式的核心：视口 (Viewport)

要让响应式生效，HTML 的 `<head>` 里必须包含这行代码，否则手机浏览器会模拟桌面宽度（通常是 980px），导致文字变得极小：

HTML

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

14.CSS性能优化

CSS 性能优化是一个多维度的课题。它的核心目标只有两个：**更快的加载（Load）和更流畅的渲染（Render）**。

以下是针对 2026 年现代 Web 标准的 CSS 性能优化指南：

一、 加载性能优化：让样式表更轻、更快

1. 压缩与合并 (Minification & Bundling)

- **压缩**：移除无用的空格、注释、换行符。使用 Vite 或 Webpack 等工具可以自动完成。
- **合并**：减少 HTTP 请求次数。但注意，在 HTTP/2 协议下，适度的文件拆分反而有利于并行下载和缓存利用。

2. 提取关键 CSS (Critical CSS)

- 将首屏内容（Above the fold）所需的样式直接内联在 HTML 的 `<style>` 标签中。
- 非关键 CSS 通过 `<link rel="preload">` 异步加载，避免阻塞页面首次渲染。

3. 减少使用 `@import`

- `@import` 会导致串行加载（必须下载完 A 才能发现并下载 B）。
- **推荐方案**：使用多个 `<link>` 标签，浏览器可以并行下载它们。

二、 选择器优化：减轻匹配压力

浏览器是从**右向左（Right-to-Left）**解析选择器的。

- **避免通配符**：`* { ... }` 会遍历每一个 DOM 节点，代价昂贵。
- **减少层级嵌套**：尽量不要写 `.nav ul li a`（浏览器要先找所有 `a`，再过滤 `li`，以此类推）。
- **推荐 BEM 命名法**：直接使用单一类名如 `.nav__item-link`，匹配速度最快。
- **避开属性选择器**：如 `[type="text"]` 的匹配效率远低于 `.input-text`。

三、 渲染性能优化：避开重排（Reflow）

重排是性能的“杀手”，它会强制浏览器重新计算页面几何结构。

1. 优先使用合成属性

动画和变换应始终优先操作以下两个属性，因为它们在 GPU 上运行，不触发重排和重绘：

- `transform` (scale, translate, rotate)
- `opacity`

2. 强制开启硬件加速

对于复杂的动画元素，使用 `will-change` 告诉浏览器提前将其提升到合成层：

CSS

```
.animating-element {
  will-change: transform, opacity;
}
```

注：不要给所有元素都加，这会过度消耗显存。

3. 使用 `content-visibility: auto`

这是现代 CSS 的黑科技。它允许浏览器跳过屏幕外（Viewport 以外）元素的渲染工作。

CSS

```
.long-list-item {
  content-visibility: auto;
  contain-intrinsic-size: 0 500px; /* 给一个预估高度，防止滚动条抖动 */
}
```

四、 布局性能：Flex vs Grid

虽然现代浏览器对二者的优化都极好，但在超大规模 DOM 环境下：

- **Flexbox**：通常在计算速度上略快于 Grid。
- **Grid**：在处理复杂的二维布局时代码更精简，减少了 HTML 层级嵌套。
- **建议**：减少 HTML 嵌套深度比纠结 Flex 还是 Grid 更重要。嵌套越深，渲染树计算越慢。

五、 属性避雷针

1. **慎用 `filter` (滤镜)**：如 `blur()` 会造成严重的滚动卡顿，因为它在绘制时计算量巨大。
2. **避免 `box-shadow` 的过度使用**：尤其是在动画元素上，阴影的计算非常耗资源。
3. **图片优化**：使用 CSS 控制图片大小时，确保图片原始尺寸与显示尺寸接近，避免浏览器实时执行高重采样算法。

15.避免重绘重排

避免重排（Reflow）和重绘（Repaint）的核心思路只有两个：**减少操作次数**和**提升操作量级**（将任务交给 GPU）。

以下是根据开发场景总结的高效避坑指南：

---

1. 样式处理：合并多次修改

浏览器为了性能，通常会有一个渲染队列，会将多次改动合并。但如果你编写不当，会强制破坏这个机制。

- **不要逐条修改样式**：

  JavaScript

  ```
  // ❌ 差：触发多次重排/重绘
  el.style.width = '100px';
  el.style.height = '100px';
  el.style.margin = '10px';

  // ☑ 好：合并为一次
  el.style.cssText = 'width: 100px; height: 100px; margin: 10px;';
  // 或者
  el.className = 'new-style';
  ```

---

2. 避免强制同步布局（Forced Synchronous Layout）

这是开发者最容易忽略的性能杀手。当你试图在修改样式后立即获取位置属性时，浏览器为了给你准确的值，必须**强制停止**当前的优化，立刻进行重排。

- **读写分离**：

  JavaScript

  ```
  // ❌ 差：读 -> 写 -> 读 -> 写（触发多次强制重排）
  for (let i = 0; i < paragraphs.length; i++) {
      paragraphs[i].style.width = box.offsetWidth + 'px';
  }

  // ☑ 好：先读再写（缓存属性）
  const width = box.offsetWidth;
  for (let i = 0; i < paragraphs.length; i++) {
      paragraphs[i].style.width = width + 'px';
  }
  ```

> **注意：** 会触发强制重排的属性包括：`offsetTop`，`offsetLeft`，`offsetWidth`，`offsetHeight`，`scrollTop`，`scrollLeft`，`clientWidth`，`getComputedStyle()` 等。

---

3. 离线操作 DOM

如果你需要对 DOM 进行大量修改（如循环插入 1000 个列表项），不要直接在渲染树上操作。

- **使用 `DocumentFragment`**：创建一个内存中的虚拟容器，把所有节点加进去，最后一次性挂载。

  JavaScript

  ```
  const fragment = document.createDocumentFragment();
  for (let i = 0; i < 100; i++) {
      const li = document.createElement('li');
      fragment.appendChild(li);
  }
  document.getElementById('list').appendChild(fragment);
  ```

- **隐藏后再修改**：先设置 `display: none`（触发一次重排），进行 100 次修改后再 `display: block`（触发一次重排）。

---

4. 动画优化：开启 GPU 加速

让动画避开主线程，直接在 GPU 中完成合成（Composite），这是性能优化的终极手段。

- **使用 `transform` 代替 `top/left`**：修改 `top` 会触发重排，修改 `transform` 只触发合成。
- **使用 `opacity` 代替 `visibility`**。
- **使用 `will-change`**：提前告诉浏览器这个元素会变，浏览器会为其创建独立的层（Layer）。

  CSS

  ```
  .animating-element {
      will-change: transform;
  }
  ```

16. 为什么vue3用proxy而不是object.defineProperty

当数据发生变化时，能够自动通知系统去更新网页。

1. 拦截能力（深度与广度）

- **Object.defineProperty**：只能拦截属性的 **get** 和 **set**。它无法监控到属性的**删除**或**新增**。这也是为什么在 Vue 2 中，直接给对象添加新属性或使用索引修改数组时，视图不会更新，必须使用 `Vue.set()`。
- **Proxy**：可以拦截多达 **13 种底层操作**，包括 `get`、`set`、`deleteProperty`（删除）、`has`（in 操作符）、`ownKeys`（遍历）等。这使得 Vue 3 能够天然支持属性的新增和删除，无需额外的 API。

2. 对数组的支持

- **Object.defineProperty**：对数组的支持非常有限。Vue 2 为了实现数组响应式，不得不手动重写（拦截）了数组的 7 个变异方法（如 `push`, `pop`, `splice` 等）。
- **Proxy**：直接支持数组的各种操作拦截。无论是通过索引修改元素，还是改变数组长度，`Proxy` 都能捕捉到变化并触发更新。

3. 初始化性能（惰性监听）

- **Object.defineProperty**：在初始化响应式数据时，Vue 2 必须递归遍历整个对象，对每一个属性调用 `Object.defineProperty`。如果对象层级很深且数据量大，会导致严重的初始化性能损耗。
- **Proxy**：是**按需（惰性）监听**的。它只对当前访问的对象层级创建 Proxy 代理。只有当你访问深层嵌套属性时，Vue 3 才会动态地为该属性创建下一层的 Proxy。这极大地提升了组件初始化的速度。

17.unknown any的区别

在 TypeScript 中，`unknown` 被称为**安全类型**，所谓“强制进行类型检查”，是指 **TypeScript 不允许你直接对 `unknown` 类型的变量执行任何操作，除非你先通过某种方式证明它确实具备该操作的能力。**

这与 `any` 形成了鲜明对比：`any` 是完全放弃检查，而 `unknown` 是“先检查，后使用”。

具体来说，这种强制检查体现在以下三个方面：

1. 禁止直接访问属性或方法

对于 `any` 类型的变量，你可以随意调用不存在的方法，TS 不会报错（但运行时会崩）。而对于 `unknown`，TS 在编译阶段就会拦截。

TypeScript

```
let valueAny: any = "hello";
let valueUnknown: unknown = "hello";

// any 不检查，编译通过，但如果运行时不是函数会报错
valueAny.toUpperCase();

// unknown 强制检查，直接编译报错：
// "对象的类型为 'unknown'"
valueUnknown.toUpperCase();
```

2. 必须先进行“类型收缩” (Type Narrowing)

为了使用 `unknown` 变量，你必须使用逻辑判断来缩小它的范围。只有当编译器确定了它的具体类型后，才允许你调用对应的方法。

TypeScript

```
let value: unknown = "hello";

if (typeof value === "string") {
  // 在这个代码块内，TS 确定 value 是 string，所以允许调用
  console.log(value.toUpperCase());
}
```

3. 禁止直接赋值给其他具体类型

`unknown` 只能赋值给 `any` 或 `unknown` 本身。如果你想把它赋值给一个具体的类型（如 `string`），必须进行强制类型断言。

TypeScript

```
let value: unknown = "hello";

// 报错：不能将类型“unknown”分配给类型“string”
let str: string = value;

// 必须通过断言告诉 TS：我确定它是字符串
let str2: string = value as string;
```

18.var let const 的区别

vue3禁用var

在 JavaScript 和 TypeScript 中，`var`、`let` 和 `const` 是声明变量的三种方式。随着 ECMAScript 6 (ES6) 的引入，推荐优先使用 `let` 和 `const` 以获得更好的代码安全性和可读性。

以下是它们的主要区别：

1. 作用域 (Scope)

作用域决定了变量在何处可以被访问。

- `var`：**函数作用域**。如果 `var` 在函数内部声明，它在整个函数内都可见；如果在函数外部声明，它是全局的。
- `let` & `const`：**块级作用域**。它们只在声明它们的代码块 `{}`（如 `if` 语句、`for` 循环或普通的 `{}` 块）内有效。

2. 变量提升 (Hoisting)

- `var`：存在变量提升。变量可以在声明之前使用，值为 `undefined`。
- `let` & `const`：虽然也会提升，但在声明之前访问会抛出错误（处于“暂时性死区”）。这强制要求必须先声明后使用，减少了潜在 Bug。

3. 重复声明 (Redeclaration)

- `var`：允许在同一作用域内多次声明同名变量。
- `let` & `const`：在同一作用域内不允许重复声明，否则会报错。

4. 值修改 (Reassignment)

- `var` & `let`：声明后可以随时重新赋值。
- `const`：声明时必须初始化，且之后不能重新赋值。

- *注意*：如果 `const` 声明的是一个对象或数组，虽然不能改变变量本身的引用，但可以修改对象内部的属性或数组的元素。

19.ES6新特性

ES6（ECMAScript 2015）是 JavaScript 的一次重大更新，它让 JS 从一门“小脚本语言”真正具备了编写大型复杂应用的能力。你在“硅谷甄选”项目中使用的大部分现代语法都源自 ES6。

以下是面试中最常考的核心新特性总结：

1. 变量声明：`let` 与 `const`

引入了块级作用域，解决了 `var` 带来的变量提升和全局污染问题。

- `let`：用于声明可变变量，具有块级作用域。
- `const`：用于声明常量，声明时必须初始化且之后不能重新赋值。

2. 箭头函数 (Arrow Functions)

提供了更简洁的函数写法，并且不绑定自己的 `this`，它会捕获其所在上下文的 `this` 值。

JavaScript

```
const add = (a, b) => a + b;
```

3. 模板字符串 (Template Literals)

使用反引号（`` ` ``）定义，支持多行字符串和变量嵌入（`${expression}`），极大地方便了字符串拼接。

4. 解构赋值 (Destructuring Assignment)

可以直接从数组或对象中提取值并赋值给变量，你在导入模块或处理接口数据时会频繁用到。

JavaScript

```
const { name, age } = obj; // 对象解构
const [first, second] = [1, 2]; // 数组解构
```

5. 展开与剩余运算符 (`...`)

- **展开 (Spread)**：将数组或对象展开，常用于合并数组或浅拷贝对象。
- **剩余 (Rest)**：用于获取函数的多余参数。

6. 类 (Classes)

引入了 `class` 关键字，虽然本质上仍是基于原型的语法糖，但让面向对象编程（OOP）的写法更加清晰直观。

7. 模块化 (ES Modules)

引入了官方的模块标准 `import` 和 `export`，支持静态编译，是 Tree Shaking 的基础。

8. Promise

异步编程的官方解决方案，解决了“回调地狱”问题。你在处理 API 请求时常用的 `then/catch` 逻辑就源于此。

9. Proxy 与 Reflect

- `Proxy`：可以拦截并自定义对象的基本操作（如 `get`、`set`），是 Vue3 响应式的核心。
- `Reflect`：提供了操作对象的更规范的 API。

10. 新的数据结构

- `Set`：类似于数组，但成员的值都是唯一的，常用于数组去重。
- `Map`：类似于对象，但“键”的范围不限于字符串，各种类型的值（包括对象）都可以当作键。

💡 面试官通常会追问：

“你认为 ES6 中最重要的特性是什么？为什么？”

建议回答：**Promise**（改变了异步处理方式）或者 **Proxy**（现代框架响应式的基石）。

20.多行省略

webkit方法

```
.ellipsis-multi {
  display: -webkit-box;            /* 必须结合的属性，将对象作为弹性伸缩盒子模型显示 */
  -webkit-box-orient: vertical;    /* 必须结合的属性，设置伸缩盒子的子元素排列方式 */
  -webkit-line-clamp: 3;           /* 限制显示的文本行数 */
  overflow: hidden;                /* 隐藏超出的内容 */
  text-overflow: ellipsis;         /* 超出部分显示省略号 */
}
```

双伪元素：::before定义省略号content，绝对定位到右下角，::after1em宽高，绝对定位到右下角，设置颜色为背景颜色

渐变块：右下角绝对定位渐变块

21.词法环境和变量环境

在 JavaScript 的**执行上下文**中，变量环境（Variable Environment）和词法环境（Lexical Environment）是两个存储变量和函数声明的“仓库”。

虽然它们在结构上非常相似，但它们分工明确，决定了 `var` 与 `let/const` 的不同行为。

---

1. 词法环境 (Lexical Environment)

词法环境是现代 JavaScript 作用域的核心。它主要负责存储 `let`、`const` 声明的变量，以及**函数声明**。

- **组成部分**：
  - **环境记录 (Environment Record)**：实际存储变量和函数的地方。
  - **对外部环境的引用 (Outer Reference)**：允许访问父级作用域（实现作用域链）。
- **特性**：
  - **暂时性死区 (TDZ)**：在创建阶段，`let` 和 `const` 虽被登记，但未被初始化。在代码运行到声明行之前，该环境禁止访问这些变量。
  - **块级作用域**：每当进入一个 `{}` 块（如 `if` 或 `for`），引擎会为该块创建一个新的词法环境。

---

2. 变量环境 (Variable Environment)

变量环境本质上也是一个词法环境，但在当前的 JavaScript 引擎中，它被专门用来存储由 `var` 声明的变量。

- **特性**：
  - **变量提升 (Hoisting)**：在创建阶段，`var` 声明的变量会被存入变量环境，并立即初始化为 `undefined`。
  - **不支持块级作用域**：`var` 不识别 `{}`，它只在函数作用域或全局作用域的变量环境中登记。

---

3. 两者的区别与联系

在 ES6 之前的规范中，并没有词法环境和变量环境的明确拆分。ES6 为了在保持向后兼容（支持 `var`）的同时引入块级作用域（支持 `let/const`），才将它们分离开来。

维度	词法环境 (Lexical Environment)	变量环境 (Variable Environment)
存储内容	`let`, `const`, 函数声明	`var` 声明
初始化状态	未初始化 (进入 TDZ)	初始化为 `undefined`
作用域	支持块级作用域 `{}`	仅支持函数/全局作用域
查找顺序	引擎首先在这里查找变量	词法环境找不到时，再来这里查找

---

4. 实例演示

观察以下代码在执行上下文**创建阶段**的内部状态：

JavaScript

```
let a = 10;
var b = 20;

if (true) {
    let c = 30;
    var d = 40;
}
```

执行上下文内部大致示意：

1. **全局执行上下文**：
   - **词法环境**：登记 `a` (uninitialized)。
   - **变量环境**：登记 `b` (undefined), `d` (undefined) —— *注意 `d` 穿透了 if 块进入了全局变量环境。*
2. **if 块运行阶段**：
   - 引擎会为 `if` 块创建一个临时的**新词法环境**来存放 `c`。

22.作用域链

就是执行上下文由内而外嵌套得到的，一层一层往外查找

在 JavaScript 中，每当代码执行时，都会创建一个**执行上下文**。每个上下文都有一个指向其**外部环境（Outer Reference）**的引用。

在 JavaScript 中，**作用域链（Scope Chain）** 是决定变量访问权限的一套规则。当你在代码中使用一个变量时，引擎会通过这根“链条”来寻找它的位置。

简单来说，它的逻辑是：**“就近原则，由内向外”**。

---

1. 作用域链是如何生成的？

作用域链的本质是**执行上下文（Execution Context）**中“词法环境”的引用集合。

每个执行上下文都有一个指向其**外部环境（Outer Reference）**的引用。当你访问一个变量时，引擎会按照以下顺序查找：

1. **当前执行上下文**：先在当前函数的“词法环境”和“变量环境”里找。
2. **父级执行上下文**：如果没找到，就顺着 `Outer Reference` 往上一层找。
3. **全局执行上下文**：一直找到最顶层的全局作用域。如果还没找到，就会报 `ReferenceError`。

---

2. 词法作用域（静态作用域）

理解作用域链的关键在于：**JavaScript 采用的是词法作用域（Static Scoping）**。

- **规则**：作用域链是由函数**定义时的位置**决定的，而不是调用时的位置。
- **后果**：无论函数在哪里被调用，它能访问哪些变量在代码写完的那一刻就已经确定了。

JavaScript

```
let value = 1;

function foo() {
  console.log(value);
}

function bar() {
  let value = 2;
  foo(); // 调用 foo
}

bar(); // 结果输出 1，而不是 2
```

> **解析**：`foo` 在定义时，它的上一级作用域是全局作用域。虽然它在 `bar` 内部被调用，但它的作用域链依然指向全局。

---

3. 闭包与作用域链

闭包（Closure） 实际上就是作用域链的一种体现。

当一个内部函数被返回并在外部执行时，它依然保留着对定义时所在作用域的引用。即便外部函数已经执行完毕、从调用栈中弹出了，这个引用的“环境”依然被保留在堆内存中，供内部函数使用。

---

4. 变量查找过程（示例）

JavaScript

```
let a = "global";

function outer() {
  let b = "outer";

  function inner() {
    let c = "inner";
    console.log(a + b + c); // 这里发生了三次查找
  }
  inner();
}
outer();
```

- **找 `c`**：在 `inner` 上下文中直接找到。
- **找 `b`**：在 `inner` 没找到，顺着 `Outer` 引用在 `outer` 上下文中找到。
- **找 `a`**：在 `inner`、`outer` 都没找到，最后在 `Global` 上下文中找到。

---

5. 总结对比

概念	核心职能
词法作用域	规定了变量在**哪里**被定义（静态阶段）。
执行上下文	规定了变量在**运行阶段**的存储环境。
作用域链	将这些环境**串联**起来，提供一套从内到外的查找机制。

23.作用域链和闭包

1. **嵌套定义**：函数 A 内部定义了函数 B。
2. **变量捕获**：函数 B 的作用域链中包含了函数 A 的环境。
3. **函数返回**：函数 A 执行完毕并返回了函数 B。
4. **内存驻留**：按理说函数 A 执行完，它的变量应该被销毁。但因为函数 B 被外部引用，且函数 B 的作用域链还牵着函数 A 的变量，垃圾回收机制（GC）就不能动这些变量。

结论：闭包 = 函数 + 该函数能够访问的词法作用域。

24.数组的方法

创建数组

```
const arr = []

const arr = new Array()

const arr = Array.of()

const arr = Array.from()

const arr = [...arg1, ...arg2]
```

数组的方法

我们可以把 JavaScript 数组常用的方法按**功能**进行分类。这样不仅好记，而且在实际开发中能快速反应出该用哪一个。

---

1. 修改原数组的方法（破坏性）

这类方法会直接改变你定义的那个数组。在 React 等强调“不可变数据”的框架中要慎用。

方法	功能	返回值
`push()`	末尾添加元素	新的数组长度
`pop()`	删除末尾元素	**被删除的元素**
`unshift()`	开头添加元素	新的数组长度
`shift()`	删除开头元素	**被删除的元素**
`splice()`	增/删/改任意位置元素	被删除元素组成的数组
`sort()`	对数组排序	排序后的原数组
`reverse()`	反转数组	反转后的原数组
`fill()`	用固定值填充数组	填充后的原数组

---

2. 不修改原数组的方法（非破坏性）

这类方法会返回一个**新数组**或一个**具体的值**，原数组完好无损。

方法	功能	返回值
`map()`	遍历并加工每个元素	加工后的**新数组**
`filter()`	过滤出符合条件的元素	过滤后的**新数组**
`slice()`	截取数组的一部分	截取出的**新数组**
`concat()`	合并两个或多个数组	合并后的**新数组**
`join()`	将数组元素连接成字符串	一个字符串
`flat()`	将嵌套数组拉平（扁平化）	拉平后的**新数组**

---

3. 查找与校验的方法

用于判断数组里有什么，或者找某个东西在哪里。

方法	功能	返回值
`find()`	查找第一个符合条件的**元素**	元素本身或 `undefined`
`findIndex()`	查找第一个符合条件的**索引**	索引值或 `-1`
`indexOf()`	查找简单值的索引	索引值或 `-1`
`includes()`	判断是否包含某个值	`true` / `false`
`some()`	只要有一个元素符合条件	`true` / `false`
`every()`	必须所有元素都符合条件	`true` / `false`

---

4. 终极利器：累加器

- `reduce()`: 将数组中的所有元素通过一个函数合并成一个单一的值（可以是数字、对象、甚至新数组）。

```
const nested = [1, [2, [3, 4]]];

console.log(nested.flat());    // [1, 2, [3, 4]] (默认只拉平一层)
console.log(nested.flat(2));   // [1, 2, 3, 4] (拉平两层)
console.log(nested.flat(Infinity)); // 无论嵌套多深，全部拉平
```

25.数组去重

在 JavaScript 中，数组去重是一个非常经典的操作。根据你的需求（是追求代码简洁，还是需要兼容旧环境），有以下几种主流实现方式：

---

1. 最推荐：使用 `Set` (ES6)

这是目前最简洁、性能也极佳的方法。`Set` 是一种不允许重复值的数据结构。

- **原理**：将数组转为 Set（自动去重），再转回数组。
- **适用场景**：基础数据类型（数字、字符串）的去重。

JavaScript

```
const original = [1, 2, 2, 3, 4, 4, 5];

// 方式 A：使用扩展运算符 [...]
const unique = [...new Set(original)];

// 方式 B：使用 Array.from()
const unique2 = Array.from(new Set(original));

console.log(unique); // [1, 2, 3, 4, 5]
```

---

2. 逻辑流：使用 `filter` + `indexOf`

如果你想理解去重的底层逻辑，这个方法最直观。

- **原理**：遍历数组，只保留那些“第一次出现”的元素。如果某个元素的当前索引不等于它第一次出现的索引，说明它是重复的。

JavaScript

```
const arr = [1, 2, 1, 3, 2, 4];

const unique = arr.filter((item, index) => {
    // 检查当前元素的索引是否等于它在数组中第一次出现的索引
    return arr.indexOf(item) === index;
});

console.log(unique); // [1, 2, 3, 4]
```

---

3. 高性能：使用 `Map` 或对象键值对

对于超大型数组，`indexOf` 在每一轮循环中都会重新扫描数组，效率较低（$O(n^2)$）。使用 `Map` 可以达到线性时间复杂度（$O(n)$）。

JavaScript

```
const arr = [1, 2, 1, 3, 2, 4];
const map = new Map();
const unique = [];

for (const item of arr) {
    if (!map.has(item)) {
        map.set(item, true); // 记录该值已出现
        unique.push(item);
    }
}
```

26.arguments为什么不是数组，如何转化为数组

这是一个非常经典的问题。在 JavaScript 中，`arguments` 被称为**类数组对象（Array-like Object）**，它虽然长得像数组，但骨子里并不是。

---

1. 为什么 `arguments` 不是数组?

虽然 `arguments` 具有数组的部分特征（比如有索引 `0, 1, 2...` 和 `length` 属性），但它并不属于 `Array` 类型。

- **原型链不同**：它的原型指向的是 `Object.prototype`，而不是 `Array.prototype`。
- **缺少方法**：它**没有**数组内置的高阶方法，如 `forEach`、`map`、`filter`、`slice` 或 `pop` 等。

> **注意**：在现代开发中，`arguments` 只存在于普通函数中。在**箭头函数**里是没有 `arguments` 的。

---

2. 如何将 `arguments` 转化为数组?

现在我们有多种方式可以把这个“冒牌货”转成真正的数组。

A. 现代最推荐：扩展运算符 (Spread Operator)

这是目前最简洁、最直观的写法。

JavaScript

```
function func() {
  const args = [...arguments]; // 转化成功
  console.log(Array.isArray(args)); // true
}
```

B. 专用工具：`Array.from()`

ES6 引入这个方法就是为了专门对付“类数组对象”的。

JavaScript

```
function func() {
  const args = Array.from(arguments);
  args.map(x => x * 2); // 现在可以使用 map 了
}
```

C. 经典方案：`Array.prototype.slice`

在 ES6 出现之前，这是程序员最常用的黑科技。通过 `call` 强行让 `arguments` 借用数组的切片方法。

JavaScript

```
function func() {
  const args = Array.prototype.slice.call(arguments);
  // 或者简写为 [].slice.call(arguments);
}
```

---

3. 终极替代方案：剩余参数 (Rest Parameters)

如果你是在编写新的代码，其实已经**不推荐使用 `arguments` 了**。ES6 提供的“剩余参数”语法可以直接让你在定义函数时就拿到一个真正的数组：

JavaScript

```
// 推荐写法
function func(...args) {
  // args 直接就是一个数组，不需要任何转化
  console.log(args.map(x => x + 1));
}
```

27.set map 数组

这是一份关于 JavaScript 中 **Set**、**Map** 和 **Array（数组）** 的全方位对比整理：

---

1. 核心含义与特征

数据结构	核心含义	存储特征	唯一性
Array (数组)	有序的值列表	按索引（0, 1...）顺序排列	允许重复
Set (集合)	无序的唯一值集合	类似数组，但值不能重复	**强制唯一**
Map (映射)	键值对的集合	类似对象，但“键”可以是任何类型	**键唯一**，值可重复

---

2. 定义方式

数据结构	定义方式	示例
Array	字面量（常用）或构造函数	`const arr = [1, 2, 2];` `const arr = new Array(1, 2);`
Set	构造函数，接收可迭代对象	`const set = new Set([1, 2, 2]);` *(结果：{1, 2})*
Map	构造函数，接收嵌套数组	`const map = new Map([['key', 'val'], [1, 'one']]);`

---

3. 常用方法对比

操作类型	Array (数组)	Set (集合)	Map (映射)
添加元素	`push()` (末尾), `unshift()` (开头)	`add(value)`	`set(key, value)`
删除元素	`pop()`, `shift()`, `splice()`	`delete(value)`	`delete(key)`
查找/读取	`arr[i]`, `indexOf()`, `find()`	`has(value)` (返回布尔值)	`get(key)`, `has(key)`
获取长度	`.length`	`.size`	`.size`
清空	`arr.length = 0`	`clear()`	`clear()`
遍历	`forEach()`, `map()`, `for...of`	`forEach()`, `for...of`	`forEach()`, `for...of`

---

4. 关键区别总结

Array vs Set

- **重复性**：数组可以有重复值（如 `[1, 1]`），Set 会自动去重（如 `new Set([1, 1])` 只有 `1`）。
- **访问速度**：在大量数据中，Set 的 `has()` 查找速度极快（O(1)），而数组的 `includes()` 或 `indexOf()` 需要遍历整个数组（O(n)）。
- **主要用途**：数组用于存储需要保持顺序的清单；Set 用于去重、成员关系判断。

Map vs Object (普通对象)

- **键的类型**：Object 的键只能是字符串或 Symbol；Map 的键可以是**任何类型**（如对象、函数、数字）。
- **有序性**：Map 严格按照插入顺序进行遍历；Object 的遍历顺序在某些情况下不可靠。
- **继承**：Object 有原型（如 `toString` 等方法），Map 是纯粹的键值对集合。

技巧

- **数组一键去重**：`const unique = [...new Set(oldArray)];`
- **数组转 Map**：`const map = new Map(arr.map(item => [item.id, item]));`
- **判断是否包含**：`set.has(val)` 永远比 `arr.indexOf(val) !== -1` 看起来更专业且运行更快。

28.git

```
git add .
git commit -m "xxx"
git log
git push
git reset -hard xxx
git branch xxx
git checkout xxx
git merge
```

29.ESM和commonJS

这是一份关于 **ES Modules (ESM)** 与 **CommonJS (CJS)** 的全方位深度对比整理：

```
const math = require('./math.js');
```

---

1. 核心语法对比

特性	CommonJS (CJS)	ES Modules (ESM)
导出关键字	`module.exports` / `exports`	`export` / `export default`
导入关键字	`require()`	`import`
文件后缀	`.js` (Node默认) / `.cjs`	`.js` (配置后) / `.mjs`

---

2. 核心区别 (重点)

A. 编译时 vs 运行时

- **ESM (静态)**：在代码执行前，编译器就能确定模块之间的依赖关系。这使得 **Tree Shaking**（剔除未使用的代码）成为可能。
- **CJS (动态)**：在代码运行到这一行时才去加载模块。因此，你可以把 `require` 放在 `if` 语句里。

B. 值的引用 vs 值的拷贝

这是面试中最常被问到的底层差异：

- **ESM (只读引用)**：导入的是原始值的**实时连接**。如果模块内部修改了变量，外部获取的值会**同步变化**。

- **CJS (值拷贝)**：导入的是值的**快照**。一旦模块被导出，其内部变量后续的修改不会影响到已经导出的值。

C. 同步 vs 异步

- **CJS**：主要是为服务器端设计，模块存放在本地磁盘，读取极快，所以采用**同步加载**。
- **ESM**：为浏览器设计，模块可能分布在网络上，因此采用**异步加载**，支持顶层 `await`。

---

3. 加载逻辑与缓存

- **缓存机制**：两者都会缓存已加载的模块。第二次加载同一个模块时，都会直接从内存读取。
- **循环依赖**：
  - **CJS**：如果出现循环引用，可能会拿到一个只加载了一部分的空对象 `{}`。
  - **ESM**：由于是静态引用，它能更好地处理循环依赖，确保能引用到最终定义的变量。

---

4. 环境兼容性

环境	CommonJS	ES Modules
浏览器	❌ 原生不支持 (需 Webpack/Vite 转换)	☑ 原生支持 (`<script type="module">`)
Node.js	☑ 原生默认支持	☑ 支持 (需 package.json 中设置 `"type": "module"`)

---

5. 无法忽视的“互操作性” (Interoperability)

在 Node.js 环境下：

1. **ESM 可以引入 CJS**：

   JavaScript

   ```
   import cjs from './lib.cjs'; // 正常工作
   ```

2. **CJS 无法通过 require 引入 ESM**：

   因为 ESM 是异步的，而 `require` 是同步的。如果要在 CJS 中使用 ESM，必须使用异步的 `import()` 函数。

   JavaScript

   ```
   // CJS 环境
   const esm = await import('./lib.mjs'); // 只能这样写
   ```

30.AJAX，Axios，fetch

在 JavaScript 的开发历程中，这三者代表了**异步通信（网络请求）**在不同时代的进化。

---

1. AJAX (Asynchronous JavaScript and XML)

身份：它不是一种新技术，而是一种**技术方案**。它的核心是浏览器内置的 `XMLHttpRequest` (XHR) 对象。

- **特点**：
  - 在不重新加载整个页面的情况下更新网页。
  - 语法非常零碎，需要处理各种状态码（`onreadystatechange`）。
  - **回调地狱**：如果请求之间有依赖关系，代码会嵌套得很深。
- **代码长相**：

  JavaScript

  ```
  open
  onreadystatechange(readyState  === 4 && xhr.status === 200)
  send
  ```

  ```
  var xhr = new XMLHttpRequest();
  xhr.open('GET', 'https://api.example.com/data', true);
  xhr.onreadystatechange = function() {
    if (xhr.readyState === 4 && xhr.status === 200) {
      console.log(xhr.responseText);
    }
  };
  xhr.send();
  ```

---

2. Fetch API

身份：它是浏览器原生提供的 **AJAX 替代者**（ES6+ 引入）。

- **特点**：
  - **基于 Promise**：写法比 XHR 简洁得多，支持链式调用。
  - **更符合语义**：它提供了通用的 `Request` 和 `Response` 对象。
  - **缺点**：
    - 默认**不带 Cookie**（需配置 `credentials`）。
    - **错误处理反直觉**：即使服务器返回 404 或 500，Fetch 也不会进入 `.catch()`，只有网络故障或请求被阻止时才会报错。你必须手动检查 `response.ok`。
    - 不支持**请求超时控制**（需配合 `AbortController`）和**上传进度监控**。
- **代码长相**：

  JavaScript

  ```
  fetch('https://api.example.com/data')
    .then(res => {
      if (!res.ok) throw new Error('Network error');
      return res.json();
    })
    .then(data => console.log(data))
    .catch(err => console.log(err));

  // 这里res是请求成功后，返回的响应对象
  // 这里的data是第一个then的返回值
  // err也是针对第一个then提出，表示响应失败的错误对象
  ```

---

3. Axios

身份：这是一个基于 Promise 的**第三方库**，在底层根据环境自动切换（浏览器用 XHR，Node.js 用 http 模块）。

- **特点（为什么大家爱用它）**：
  - **拦截器 (Interceptors)**：在请求发送前（加 Token）或响应返回后（统一处理错误）进行拦截，非常强大。
  - **自动转换 JSON**：它会自动帮你把响应数据转成 JS 对象。
  - **防御 CSRF**：内置了一些安全保护。
  - **支持取消请求**和**进度监控**。
  - **错误处理更友好**：4xx/5xx 的状态码会直接进入失败逻辑。
- **代码长相**：

  JavaScript

  ```
  axios.get('https://api.example.com/data')
    .then(res => console.log(res.data))
    .catch(err => console.log(err));
  ```

31.JSON

简单来说，**JSON**（JavaScript Object Notation）是一种轻量级的**数据交换格式**。它基于文本，易于人类阅读和编写，同时也易于机器解析和生成。

虽然名字里带有 "JavaScript"，但 JSON 是一种**独立于语言**的文本格式，几乎所有的现代编程语言（Python, Java, PHP, Go 等）都支持它。

---

1. JSON 的基本语法规则

JSON 的格式与 JavaScript 的对象字面量非常相似，但规则更严格：

- **复合类型**：
  - **对象**：使用花括号 `{}` 包裹，存储键值对 `{"key": "value"}`。
  - **数组**：使用方括号 `[]` 包裹，存储有序的值列表 `["a", "b"]`。
- **基本规则**：
  - **键（Key）必须用双引号 `""` 包裹**（这是与 JS 对象最大的区别，JS 对象的键可以不加引号）。
  - **字符串值必须用双引号 `""`**，不能用单引号。
  - 数据之间用逗号 `,` 分隔，但**末尾不能有逗号**。

---

2. JSON 支持的数据类型

JSON 只能存储以下几种简单数据类型：

1. **字符串**：必须双引号，如 `"Hello"`。
2. **数字**：整数或浮点数，如 `42` 或 `3.14`。
3. **布尔值**：`true` 或 `false`。
4. **空值**：`null`。
5. **对象**：嵌套的对象。
6. **数组**：值的集合。

> **注意**：JSON **不能**存储函数（Function）、日期（Date）对象或未定义（Undefined）。

---

3. 为什么大家都用 JSON?

在 JSON 流行之前，互联网上主要使用 **XML** 传输数据。

- **更简洁**：相比于 XML 繁琐的标签（`<name>吴悠</name>`），JSON 的 `{"name": "吴悠"}` 占用字节更少，传输更快。
- **原生支持**：在 JavaScript 中，你可以直接把 JSON 字符串转成对象，操作极其方便。

---

4. JavaScript 中的两个核心方法

在前端开发中，你最常处理的就是 JSON 字符串与 JS 对象之间的转换：

A. JSON.parse() —— “反序列化”

将从服务器拿到的 **JSON 字符串** 变成 **JS 对象**。

JavaScript

```
const jsonStr = '{"name": "吴悠", "age": 18}';
const obj = JSON.parse(jsonStr);
console.log(obj.name); // "吴悠"
```

B. JSON.stringify() —— “序列化”

将 **JS 对象** 变成 **JSON 字符串**，以便通过网络发送给后端。忽略值为undefined的字段，把undefine变为null

JavaScript

```
const user = { name: "Gemini", age: 1 };
const jsonString = JSON.stringify(user);
// 结果: '{"name":"Gemini","age":1}'
```

32.事件发布订阅

```
class EventEmitter {
  // 使用一个对象（Object）或映射（Map），键（Key）是事件名，值（Value）是一个数组，用来存放该事件下的所有回调函数。
  constructor() {
    this.events = {}; // 存储事件和对应的回调函数列表
  }

  // 1. 订阅
  on(eventName, callback) {
    if (!this.events[eventName]) {
      this.events[eventName] = [];
    }
    // 存放该事件的回调函数
    this.events[eventName].push(callback);
  }

  // 2. 发布
  // 找到对应事件名下的所有函数，并依次执行它们，同时传递参数
  emit(eventName, ...args) {
    // 订阅的时候回调函数存的就是一个数组
    const callbacks = this.events[eventName];
    if (callbacks) {
```

```
      callbacks.forEach(cb => cb(...args));
    }
  }

  // 3. 取消订阅
  // 找到事件对应的数组，利用 filter 或 splice 将特定的函数剔除
  // 只是不订阅这个事件了
  off(eventName, callback) {
    const callbacks = this.events[eventName];
    if (callbacks) {
      this.events[eventName] = callbacks.filter(cb => cb !== callback);
    }
  }
}

// 使用示例
const bus = new EventEmitter();
bus.on('get_data', (data) => console.log('收到数据:', data));
bus.emit('get_data', { name: 'Gemini' }); // 打印：收到数据: {name: "Gemini"}
```

33.遍历对象属性

方法	自身属性	继承属性（原型）	不可枚举属性	Symbol 属性
`for...in`	☑	☑	✕	✕
`Object.keys()`	☑	✕	✕	✕
`Object.getOwnPropertyNames()`	☑	✕	☑	✕
`Object.getOwnPropertySymbols()`	☑	✕	✕	☑
`Reflect.ownKeys()`	☑	✕	☑	☑

1. `Object.getOwnPropertySymbols()` 方法返回一个给定对象自身的 **所有 Symbol 属性的数组**

34.判断对象是否具有属性

在 JavaScript 中，判断一个对象是否拥有某个属性，主要取决于你是想查找**对象自身的属性**，还是包括**原型链上的继承属性。**

以下是四种最常用的判断方法：

---

1. `in` 操作符（包含继承属性）

`in` 会检查属性是否存在于对象中，**包括它原型链上的属性。**

- **特点**：只要对象能访问到该属性，就返回 `true`。
- **适用场景**：不介意属性是继承来的，只要“有”就行。

JavaScript

```
const obj = { name: '吴悠' };
console.log('name' in obj);      // true
console.log('toString' in obj);  // true (继承自 Object 原型)
```

---

2. `hasOwnProperty()` 方法（仅限自身属性）

这是最经典的方法，它只会检查属性是否直接存在于**对象自身**上。

- **特点**：忽略原型链。
- **注意**：如果对象是用 `Object.create(null)` 创建的，它没有原型，也就没有这个方法，调用会报错。

JavaScript

```
const obj = { name: '吴悠' };
console.log(obj.hasOwnProperty('name'));     // true
console.log(obj.hasOwnProperty('toString')); // false
```

---

3. `Object.hasOwn()` （现代标准，最推荐）

这是 ES2022 引入的新方法，旨在取代 `hasOwnProperty()`。

- **优点**：它是静态方法，即使对象没有原型（如 `Object.create(null)` ）或者 `hasOwnProperty` 被覆盖了，它依然能稳定工作。
- **适用场景**：现代项目开发的首选。

JavaScript

```
const obj = Object.create(null);
obj.age = 18;

// console.log(obj.hasOwnProperty('age')); // 报错
console.log(Object.hasOwn(obj, 'age'));    // true
```

总结对照表

方法	自身属性	继承属性	建议
`in`	☑	☑	需要考虑原型链时使用
`hasOwnProperty`	☑	❌	传统写法，注意 `null` 原型对象
`Object.hasOwn`	☑	❌	**最推荐，最安全**
`obj.prop !== undefined`	☑	☑	简单但不严谨（无法区分属性值为 undefined 的情况）

35.深浅拷贝

在 JavaScript 中，拷贝是指将一个对象的值复制到另一个变量中。根据复制的“深度”不同，分为**浅拷贝**和**深拷贝**。

---

1. 浅拷贝 (Shallow Copy)

浅拷贝只复制对象的第一层属性。如果属性是**基本类型**（如数字、字符串），拷贝的是值；如果属性是**引用类型**（如对象、数组），拷贝的是**内存地址**。

- **后果**：修改拷贝后对象的子对象，原对象的子对象**也会跟着改变**，因为它们指向同一个地方。
- **常用方法**：
  - `Object.assign({}, obj)`
  - 扩展运算符 `[...arr]` 或 `{...obj}`
  - 数组的 `Array.prototype.slice()` 或 `concat()`

JavaScript

```
const original = { name: '吴悠', info: { age: 18 } };
const shallow = { ...original };

shallow.info.age = 20;
console.log(original.info.age); // 20 (被污染了！)
```

---

2. 深拷贝 (Deep Copy)

深拷贝会递归地复制对象的所有层级。它会在内存中开辟一块全新的空间，将原对象完全克隆一份。

- **后果**：拷贝后的对象与原对象完全隔离，修改互不影响。
- **常用方法**：

A. `JSON.parse(JSON.stringify(obj))`

这是最简单的“一行代码”方案。

- **缺点**：会丢失函数、`undefined`、`Symbol`，且无法处理循环引用（对象自己引用自己）。

B. `structuredClone(obj)` (现代标准推荐)

这是浏览器原生提供的深拷贝方法，比 JSON 方案更强大。

- **优点**：支持 `Date`, `RegExp`, `Map`, `Set` 等。
- **缺点**：依然不支持克隆函数。

C. 手写递归或使用第三方库（如 Lodash 的 `_.cloneDeep`）

这是最稳健的方案，可以处理复杂的循环引用和特殊类型。

36.freeze属性

无法实现对象的增删改，以及对象原型的修改

在 JavaScript 中，`Object.freeze()` 是一个用于**彻底冻结**对象的方法。一旦一个对象被冻结，它就变成了一个不可变的“冰块”，你无法对其进行任何修改。

---

1. 冻结后会发生什么？

当你对一个对象执行了 `Object.freeze(obj)` 后，该对象会产生以下变化：

- **无法添加新属性**：不能给对象增加原来没有的键。
- **无法删除现有属性**：`delete obj.prop` 会失效。
- **无法修改属性值**：现有的属性值不能被重新赋值。
- **无法修改属性描述符**：不能通过 `Object.defineProperty` 改变属性的可枚举性、可配置性等。
- **原型不可变**：你不能更改该对象的原型。

JavaScript

```
const user = { name: "吴悠" };
Object.freeze(user);

user.name = "Gemini"; // 静默失败（严格模式下抛出 TypeError）
user.age = 18;        // 无法添加
delete user.name;     // 无法删除

console.log(user.name); // 依然是 "吴悠"
```

---

2. 核心特性：浅冻结 (Shallow Freeze)

这是最需要注意的一点：`Object.freeze` **是浅层的。**

它只能冻结对象的第一层属性。如果属性值是另一个对象，那么那个嵌套对象**依然是可以修改的。**

例子：

JavaScript

```
const config = {
  version: "1.0",
  details: {
    author: "吴悠"
  }
};

Object.freeze(config);

config.version = "2.0";       // 无效
config.details.author = "Gemini"; // 有效！内部对象没被冻结
```

---

3. 如何实现“深冻结”？

如果你需要完全保护一个复杂对象，需要递归地冻结每一个属性：

JavaScript

```
function deepFreeze(obj) {
  // 获取所有属性名
  const propNames = Object.getOwnPropertyNames(obj);
  // 在冻结自身之前，先冻结每个嵌套对象
  propNames.forEach(name => {
    const prop = obj[name];
    if (typeof prop === 'object' && prop !== null) {
      deepFreeze(prop);
    }
  });
  return Object.freeze(obj);
}
```

---

4. 相似方法的对比

JavaScript 提供了三种逐渐加强的限制方法：

方法	无法添加属性	无法删除属性	无法修改值
`Object.preventExtensions()`	☑	✕	✕
`Object.seal()` **(密封)**	☑	☑	✕
`Object.freeze()` **(冻结)**	☑	☑	☑

---

5. 如何检测一个对象是否被冻结?

你可以使用 `Object.isFrozen(obj)` 来判断:

JavaScript

```
const obj = { a: 1 };
console.log(Object.isFrozen(obj)); // false
Object.freeze(obj);
console.log(Object.isFrozen(obj)); // true
```

37.箭头函数和普通函数区别

在 JavaScript 中，箭头函数（Arrow Function）与普通函数（Regular Function）不仅仅是语法的简写，它们在行为逻辑上有着本质的区别。

以下是它们最核心的 5 个区别:

---

1. `this` 的指向（最本质区别）

- **普通函数**: `this` 是**动态绑定**的。它指向"调用该函数的那个对象"。谁调用它，`this` 就指向谁。
- **箭头函数**: 没有自己的 `this`。它的 `this` 是**静态绑定**的，会捕获其**定义时**所在上下文的 `this` 值。即使使用 `call()`、`apply()` 或 `bind()`，也无法改变箭头函数的 `this` 指向。

---

2. 构造函数能力

- **普通函数**: 可以作为**构造函数**。可以使用 `new` 关键字创建实例。
- **箭头函数**: **不能**作为构造函数。尝试对箭头函数使用 `new` 会抛出 `TypeError`。因为它没有 `[[Construct]]` 方法，也没有 `prototype` 属性。

---

3. `arguments` 对象

- **普通函数**: 内部可以使用 `arguments` 对象，这是一个类数组对象，包含了函数接收到的所有参数。
- **箭头函数**: **没有**自己的 `arguments`。如果你在箭头函数内访问它，拿到的是外层函数的 `arguments`。

> **建议**: 在箭头函数中，请使用 ES6 的**剩余参数** `(...args) => {}` 来代替 `arguments`。

---

4. 原型属性 (`prototype`)

- **普通函数**: 拥有 `prototype` 属性。
- **箭头函数**: 没有 `prototype` 属性。

---

5. 语法与返回值

- **普通函数**：必须使用 `return` 关键字才能返回值。
- **箭头函数**：支持**隐式返回**。如果函数体只有一行表达式且没有大括号，它会自动返回该表达式的结果。

JavaScript

```
// 普通函数
const add = function(a, b) {
  return a + b;
};

// 箭头函数（隐式返回）
const addArrow = (a, b) => a + b;
```

总结对比表

特性	普通函数	箭头函数
`this` **指向**	动态，取决于调用者	静态，继承自定义时的上下文
`new` **构造**	支持	不支持
`arguments`	拥有自己的	借用外层的（建议用 `...args`）
`prototype`	有	无
作为对象方法	推荐（`this` 指向该对象）	不推荐（`this` 可能指向全局）

38.事件委托

父代子责

事件委托（Event Delegation） 是 JavaScript 中一种极其重要的性能优化技巧。它利用了 DOM 的**事件冒泡（Event Bubbling）机制，将子元素的事件监听器绑定到它的父级元素**（或更上层的祖先元素）上。

---

1. 核心原理：冒泡机制

当一个 DOM 元素触发某个事件（如 `click`）时，这个事件并不会只停留在该元素上，而是会像水底的气泡一样，一层一层向上传递给父元素，直到 `document` 甚至 `window`。

事件委托的做法是： 不在每个子节点上单独设置监听器，而是只在父节点上设一个。当事件冒泡到父节点时，父节点通过检查“是谁触发了事件”来决定如何处理。

---

2. 为什么要用事件委托?

A. 节省内存（性能优化）

如果你有一个包含 1000 个列表项（`<li>`）的列表，给每个 `<li>` 都绑定一个 `click` 事件会消耗大量内存。使用事件委托，你只需要给父级 `<ul>` 绑定 **1 个** 监听器即可。

B. 动态元素管理

这是最实用的优点。如果你通过 JavaScript 动态添加了新的 `<li>`，你**不需要**再为新元素重新绑定事件。因为点击新元素产生的事件依然会冒泡到已经绑定了监听器的父元素上。

---

3. 代码实现

我们要判断到底是谁被点击了，通常使用事件对象中的 `event.target`。

JavaScript

```
// 假设 HTML 结构为 <ul id="parent"> <li>item 1</li> ... </ul>
const parent = document.getElementById('parent');

parent.addEventListener('click', function(event) {
  // event.target 是真正被点击的那个子元素
  if (event.target.tagName === 'LI') {
    console.log('你点击了列表项：', event.target.innerText);
  }
});
```

---

4. **关键概念对比**

属性	含义	在事件委托中的角色
`event.target`	**触发**事件的源头元素。	那个被点击的子元素（如 `<li>`）。
`event.currentTarget`	**当前绑定**监听器的元素。	代理人（如父元素 `<ul>`）。

---

5. 局限性

并不是所有事件都能委托：

- **不支持冒泡的事件**无法委托：例如 `focus`、`blur`、`mouseenter`、`mouseleave`。
- **过度委托**：如果把所有事件都委托给 `document`，可能会导致逻辑过于复杂，影响调试。

39.状态码

HTTP 状态码是前后端沟通的“黑话”。为了让系统既标准又灵活，通常会结合 **HTTP 标准状态码** 和 **后端自定义业务状态码** 共同使用。

一、 HTTP 标准状态码（Standard Codes）

这些由 RFC 协议定义，浏览器、CDN 和代理服务器都能看懂。

1xx 信息性（Informational）

- **101 Switching Protocols**：切换协议。常见于 HTTP 升级为 **WebSocket**。

2xx 成功（Success）

- **200 OK**：请求成功，最常见。
- **201 Created**：资源创建成功（如注册成功、发布博客成功）。
- **204 No Content**：请求成功但没有返回内容（常见于删除成功）。

3xx 重定向（Redirection）

- **301 Moved Permanently**：永久重定向。旧网址废弃，SEO 权重转移。
- **302 Found**：临时重定向。比如未登录时跳转到登录页。
- **304 Not Modified**：**协商缓存命中**。服务器告诉浏览器："文件没变，直接用你本地的吧"。

4xx 客户端错误（Client Error）

- **400 Bad Request**：参数错误。后端校验没通过。
- **401 Unauthorized**：未登录或 Token 失效。
- **403 Forbidden**：已登录但**没权限**（比如普通用户尝试删除管理员数据）。
- **404 Not Found**：资源不存在。
- **405 Method Not Allowed**：方法不对（接口要 POST，你用了 GET）。

5xx 服务器错误（Server Error）

- **500 Internal Server Error**：后端代码报错了。
- **502 Bad Gateway**：网关错误。通常是 Nginx 找不到后端服务了。
- **504 Gateway Timeout**：网关超时。后端处理太慢，Nginx 等不及了。

二、 后端自定义状态码（Business Codes）

为什么要自定义？

HTTP 状态码有限（比如 400 只能代表参数错误），但实际业务中"错误"的原因千奇百怪：余额不足、验证码过期、账号被封禁... 这些都需要通过自定义状态码来精准告知前端。

1. 常见的自定义响应结构

后端通常会返回一个 JSON 对象，里面包含业务状态码：

JSON

```
{
  "code": 20001,            // 自定义业务状态码
  "message": "验证码已过期", // 提示信息
  "data": null              // 实际数据
}
```

2. 自定义状态码的分段规则

大厂通常会按功能模块对 code 进行分段，方便定位问题：

code 范围	模块含义	举例
10000 - 19999	**通用/系统类**	10001 (系统维护中), 10002 (参数非法)
20000 - 29999	**用户/权限类**	20005 (手机号已注册), 20008 (账号被封禁)
30000 - 39999	**订单/交易类**	30001 (余额不足), 30005 (库存告急)
40000 - 49999	**第三方服务类**	40001 (微信支付接口超时), 40002 (短信平台欠费)

40.cookie session jwt

这三个概念都是为了解决我们之前聊过的 **"HTTP 无状态"** 问题。它们的目标一致：让服务器记住"你是谁"。

我们可以把它们看作三种不同的"身份验证演进史"。

---

1. Cookie：存储在浏览器的“小纸条”

Cookie 并不是一种独立的验证机制，它其实是一个**存储容器**。

- **原理**：服务器通过响应头 `Set-Cookie` 发送一段文本，浏览器会自动把它存起来。下次请求时，浏览器会自动在请求头带上这段文本发回给服务器。
- **特点**：
  - **自动携带**：只要域名匹配，浏览器会自动加上。
  - **不安全**：容易受到 CSRF（跨站请求伪造）攻击，且用户可以手动修改。
  - **容量小**：通常限制在 4KB 左右。

---

2. Session：服务器的“账本”

Session 是基于 Cookie 实现的经典验证方案。

- **工作流程**：
  1. 用户登录，服务器验证通过。
  2. 服务器在内存或数据库里开辟一块空间（账本），记录用户信息，并生成一个唯一的 `Session ID`。
  3. 服务器通过 Cookie 把这个 `Session ID` 发给浏览器。
  4. 下次请求，服务器拿着 ID 去账本里查，查到了就是“已登录”。
- **缺点**：
  - **占内存**：用户多了，服务器内存压力大。
  - **多台服务器同步难**：如果你的请求第一次发到了 A 服务器，第二次发到了 B 服务器，B 可能没记录你的 Session（需要做 Session 共享）。

---

3. JWT (JSON Web Token)：自证明的“通行证”

JWT 是目前主流的**无状态验证**方案，它不需要服务器存账本。

- **结构**：由三部分组成：`Header`（头部）、`Payload`（负载，存用户 ID 等）、`Signature`（签名，防止篡改）。
- **工作流程**：
  1. 用户登录，服务器验证通过。
  2. 服务器把用户信息（比如 `userId: 1`）用密钥加密成一个长字符串（Token），发给前端。
  3. 前端通常存在 `localStorage` 里。
  4. 下次请求，前端手动把 Token 放在 Header 里的 `Authorization` 字段发给服务器。
  5. **服务器不查库**，直接用密钥“解密”验证签名是否合法。合法就通过。
- **优点**：
  - **扩展性极强**：由于服务器不存数据，哪怕有 100 台服务器，只要密钥一样，都能验证。
  - **跨域友好**：不受 Cookie 同源策略限制。

总结对比

特性	Cookie + Session	JWT (Token)
存储位置	服务器存数据，客户端存 ID	服务器不存数据，客户端存全量 Token
占用资源	随用户量增加消耗服务器内存	消耗 CPU（加密解密）
安全性	需防范 CSRF	需防范 XSS（Token 被窃取）
适用场景	中小型单体架构 Web 应用	现代微服务、移动端 App、跨域架构
时效性	服务器可随时手动注销 Session	一旦发出，除非过期，服务器很难主动使其失效

核心点拨

- **Session** 是"**以空间换时间**"：服务器存数据，查询快，但占内存。
- **JWT** 是"**以时间换空间**"：服务器不存数据（省空间），但每次都要加解密计算（耗时间）。

41.http协议工作过程

HTTP 协议的工作过程是一个典型的**请求-响应（Request-Response）**模型。你可以把它想象成在餐厅点餐：你（客户端）看菜单并下单，服务员（网络传输）把订单送到厨房，厨师（服务器）做菜并由服务员端回给你。

在技术层面，这个过程通常分为以下 **5 个核心步骤**：

---

1. 域名解析 (DNS Lookup)

当你输入 `https://www.example.com` 时，浏览器并不知道这台服务器在哪。它首先会向 **DNS 服务器**询问该域名对应的 **IP 地址**。

- **过程**：浏览器检查缓存 → 操作系统检查 Hosts 文件 → 请求本地及远程 DNS 服务器。
- **结果**：拿到类似 `192.0.2.1` 的物理地址。

---

2. 建立 TCP 连接 (Three-way Handshake)

拿到 IP 后，客户端需要和服务器建立一条可靠的通信链路。这就是著名的 **TCP 三次握手**：

1. **SYN**：客户端发送"我想连接"的信号。
2. **SYN-ACK**：服务器回复"收到，我也准备好了"。
3. **ACK**：客户端回复"好的，那我们开始吧"。

> **如果是 HTTPS**：在 TCP 连接建立后，还会进行 **SSL/TLS 握手**，通过非对称加密交换密钥并验证服务器证书。

---

3. 发送 HTTP 请求 (HTTP Request)

连接修好后，浏览器会构造一个 **请求报文** 发送给服务器。一个标准的请求包含三部分：

- **请求行**：方法（GET/POST）、路径（/index.html）、协议版本（HTTP/1.1）。
- **请求头 (Headers)**：包含宿主（Host）、浏览器类型（User-Agent）、期望的数据格式（Accept）等。
- **请求体 (Body)**：如果是 POST 请求，这里会携带表单数据或 JSON。

---

4. 服务器处理并返回响应 (HTTP Response)

服务器接收到请求后，由 Web 服务器（如 Nginx 或 Node.js）解析，处理业务逻辑（如查询数据库），然后返回一个 **响应报文**：

- **状态行**：协议版本、**状态码**（200 OK, 404 Not Found 等）。
- **响应头**：服务器类型、数据类型（Content-Type）、**Cookie 设置**（Set-Cookie）等。
- **响应体**：请求的实际内容，如 HTML 代码、图片数据或 JSON 接口数据。

---

5. 浏览器渲染与连接关闭

1. **解析渲染**：浏览器接收到 HTML 后，开始解析 DOM 树，发现需要 CSS 或 JS 时，会**再次发起 HTTP 请求**获取这些资源。
2. **连接处理**：
    - **短连接**：任务完成后立即断开 TCP 连接。
    - **持久连接 (Keep-Alive)**：在 HTTP/1.1 中，连接会保持一段时间，以便复用该通道传输剩下的图片和脚本。

42.浏览器中输入url发生了什么

没错，你观察得很敏锐！刚才说的 **HTTP 工作过程** 确实就是 **“从输入 URL 到页面显示”** 这个经典面试题的核心部分。

不过，如果是在面试或者深入讨论这个话题，通常还需要加上 **“浏览器渲染”** 这个下半场。如果把 HTTP 请求比作“买建材”，那么渲染过程就是“盖房子”。

完整的全过程可以梳理为以下 **6 个阶段**：

第一阶段：导航与建连（网络传输）

这部分就是我们刚才聊的 HTTP 核心流程：

1. **DNS 解析**：把域名变成 IP。
2. **TCP 连接**：三次握手。
3. **TLS 握手**：如果是 HTTPS，建立安全加密通道。
4. **发送请求**：浏览器发送 HTTP Request。
5. **接收响应**：服务器返回 HTML 源码。

第二阶段：解析 HTML（构建骨架）

浏览器拿到 HTML 字符串后，不能直接显示，得先“读懂”它：

- **构建 DOM 树**：浏览器自上而下解析 HTML 标签，转化成树状结构的 **DOM Tree**。
- **构建 CSSOM 树**：解析下载好的 CSS 文件，生成 **CSSOM Tree**。

第三阶段：渲染树构建（合成图纸）

浏览器把 **DOM 树** 和 **CSSOM 树** 结合在一起，生成 **渲染树（Render Tree）**。

- 这一步会过滤掉那些不需要显示的东西（比如 `display: none` 的元素，或者 `<head>` 里的标签）。

第四阶段：布局 Layout（测量尺寸）

有了图纸，浏览器需要计算每个元素在屏幕上的确切位置和大小。

- 这个过程也叫 **Reflow（回流/重排）**。所有的相对测量值（如 `100%` 或 `em`）都会在这个阶段转换成绝对的像素点。

第五阶段：绘制 Painting（涂色）

浏览器根据渲染树和布局信息，调用显卡的 API，将每个像素点绘制在屏幕上。

- 这包括文本、颜色、图像、边框和阴影等。

第六阶段：合成 Compositing（分层合并）

为了性能，现代浏览器会把页面分成很多“层”（Layers）。

43.https工作过程

比http多加了一步TLS握手

HTTPS 的工作过程实际上是 **HTTP 协议**与 **SSL/TLS 协议**的结合。它的核心目标是在不安全的网络环境中，建立一个加密的、可验证身份的通信管道。

一旦连接建立，双方开始用对称加密传输数据。HTTPS 在每一段加密数据（Record）里都加入了一个东西，叫 **MAC (Message Authentication Code，消息认证码)**。

整个过程可以分为四个主要阶段：

---

1. TCP 三次握手

在进行任何加密操作之前，客户端（浏览器）必须先与服务器建立物理连接。

- 客户端发送 **SYN**。
- 服务器返回 **SYN-ACK**。
- 客户端返回 **ACK**。

---

2. SSL/TLS 握手（最核心步骤）

连接建立后，双方开始进行“安全对话”，目的是**验证服务器身份**并**协商加密密钥**。

1. **Client Hello**：客户端发送它支持的 TLS 版本、加密算法列表以及一个**随机数 (Random1)**。
2. **Server Hello**：服务器确认加密算法，并发送自己的**数字证书**以及另一个**随机数 (Random2)**。
3. **证书校验**：浏览器验证证书的合法性（是否过期、是否由受信任的 CA 机构颁发）。
4. **预主密钥 (Pre-Master Secret)**：验证通过后，客户端生成第三个**随机数 (Pre-master secret)**，并用证书里的**服务器公钥**加密后发给服务器。
5. **生成会话密钥**：服务器用自己的**私钥**解开密文，得到第三个随机数。此时，双方都拥有了 Random1、Random2 和 Pre-master secret。
    - 双方根据这三个随机数，利用相同的算法计算出同一个**对称密钥（会话密钥）**。

---

3. 数据传输（对称加密阶段）

握手完成后，非对称加密的使命就结束了。

- 接下来所有的 HTTP 请求和响应内容，都使用刚才协商好的**对称密钥**进行加解密。

- **原因**：对称加密的速度比非对称加密快几个数量级，适合传输大量网页数据。

---

4. 连接结束

当数据传输完毕，双方会发送通知并关闭 TLS 隧道，随后通过 TCP 四次挥手释放网络连接。

💡 为什么 HTTPS 是安全的？

HTTPS 的巧妙之处在于它结合了两种加密方式的优点：

- **非对称加密**：用于在握手阶段**安全地交换**对称密钥，解决了“密钥如何传给对方而不被截获”的问题。
- **对称加密**：用于在传输阶段**高效地加密**实际内容，保证了性能。
- **数字证书**：解决了“对方是谁”的问题，防止黑客伪造服务器（中间人攻击）。

44.TLS证书

它是由受信任的第三方机构——**CA（Certificate Authority，数字证书认证机构）** 颁发给网站的。它的存在主要解决两个问题：**数据加密**和**身份证明**。

---

1. SSL 证书里都有什么？

你可以把它想象成一张电子名片，包含了以下核心信息：

- **证书持有者的域名**（比如 `google.com`）。
- **证书持有者的公钥**（这是实现非对称加密的关键）。
- **颁发机构的签名**（证明这张“身份证”不是伪造的）。
- **有效期限**（起始日期到过期日期）。
- **证书颁发机构（CA）的名称**。

2. SSL 证书的两大核心作用

A. 证明“你是你”（身份鉴别）

当你访问一个银行网站时，SSL 证书会告诉你的浏览器：“别担心，这真的是工商银行的服务器，不是黑客搭的假网站。” 如果证书是伪造的、过期的或者颁发机构不受信任，浏览器就会弹出红色的**“您的连接不是私密连接”**警告。

B. 提供“加密钥匙”（数据保护）

证书里存着服务器的**公钥**。浏览器利用这个公钥，在 HTTPS 握手阶段与服务器安全地协商出一个“对称密钥”。有了这个密钥，你们之间传输的所有账号密码、银行卡号都会变成乱码。

45.TLS握手

TLS（传输层安全性协议）握手是建立 HTTPS 连接中最关键的一步。它的目标是：**验证服务器身份**、**达成加密算法共识**，并**生成用于加密数据的对称密钥**。

目前主流使用的是 **TLS 1.2**（经典且详细）和 **TLS 1.3**（现代且极简）。我们先以最经典的 TLS 1.2 为例来讲解这个过程。

TLS 1.2 握手过程（“四次握手”）

整个过程可以看作是浏览器和服务器在正式说“悄悄话”之前的一场深度“试探”：

1. 客户端问候 (Client Hello)

客户端向服务器发起请求，提供以下信息：

- **支持的 TLS 版本。**
- **加密算法套件列表**（Cipher Suites）。
- **随机数 1 (Client Random)**：这是后续生成密钥的关键原料。

2. 服务器问候与证书 (Server Hello)

服务器收到后，给回响应：

- **确认 TLS 版本和加密算法**（从客户端的列表中选一个）。
- **随机数 2 (Server Random)。**
- **服务器数字证书**：包含服务器的**公钥**和权威机构（CA）的签名。
- *(可选) 服务器密钥交换*：如果算法需要额外的参数。

3. 客户端验证与密钥交换

客户端拿到证书后，会去系统中查找受信任的 CA 列表，验证证书的真伪。验证通过后：

- **生成随机数 3 (Pre-Master Secret)**：这是一个预主密钥。
- **公钥加密**：客户端用证书里的**服务器公钥**将这个 Pre-Master Secret 加密，发送给服务器。此时，只有拥有私钥的服务器能解开它。

4. 生成最终的会话密钥 (Session Key)

此时，双方手里都有了三个材料：**Client Random + Server Random + Pre-Master Secret。**

- 双方各自利用这三个材料，通过相同的算法算出同一个**对称密钥（会话密钥）**。
- **Finished**：双方互相发一个加密的消息，确认以后所有的通话都用这个“会话密钥”加密。

TLS 1.3 的进化：更简单、更安全

由于 TLS 1.2 需要往返两个回合（2-RTT）才能完成握手，延迟较高。**TLS 1.3** 进行了极致优化：

- **1-RTT 握手**：客户端在第一次握手时，就直接把自己猜测服务器会用的算法和对应的**公钥份额**（Key Share）发过去。
- **合并步骤**：服务器在回复时直接给出自己的公钥份额。双方一次往返就能算出密钥。
- **更安全**：删除了已经被证明不安全的旧算法（如 MD5, SHA-1），且握手过程中的大部分信息都是加密的。

46.https在劫持之后为什么能不被篡改

这是一个非常经典的问题。很多人理解 HTTPS 可以**防止偷看**（加密），但不太理解它为什么能**防止篡改**（完整性）。

即使黑客（中间人）截获了你的 HTTPS 数据包，他依然无法篡改内容，核心在于 HTTPS 穿了两层“防弹衣”：**数字签名校验** 和 **MAC 消息认证码。**

---

1. 第一关：握手阶段的“身份防伪”

在建立连接时，黑客如果想篡改服务器发给你的公钥（换成他自己的公钥，从而解密你的数据），他必须伪造 **SSL 证书。**

- **为什么改不了**：SSL 证书上有 **CA 机构的数字签名。**
- **校验原理**：浏览器会用 CA 的公钥去解密这个签名，并对证书内容进行哈希运算。如果黑客改了证书里的任何一个字节（比如换了公钥），哈希值就对不上了，浏览器会立即弹出大红色的安全警告，阻断连接。

---

2. 第二关：传输阶段的“封条” (MAC)

一旦连接建立，双方开始用对称加密传输数据。HTTPS 在每一段加密数据（Record）里都加入了一个东西，叫 **MAC (Message Authentication Code，消息认证码)**。

你可以把它理解为每段数据的**“数字封条”**：

1. **发送方**：将“原始数据 + 只有双方知道的对称密钥”进行哈希计算，得到一个 MAC 值，连同加密后的数据一起发出去。
2. **黑客截获**：黑客虽然看不懂内容，但他可以尝试乱改几个字节。
3. **接收方**：收到后，用同样的密钥对数据再算一遍 MAC。
    - **结果**：只要数据被动过一个比特，算出来的 MAC 就会和传过来的 MAC 完全不同。
    - **动作**：浏览器会发现“封条”被动过，直接丢弃该数据包，判定连接不安全。

---

3. 第三关：防止“调包” (防重放攻击)

黑客可能会想：*“我改不了你的内容，那我把你发过的包原封不动地再发一次总行吧？”*（比如重复发一次转账请求）。

- **解决方法**：HTTPS 的记录层协议中包含了一个**序列号 (Sequence Number)**。
- **原理**：每一段数据都有一个单调递增的编号。双方都会记录当前该收到几号包。如果黑客重发旧包，或者删掉其中一个包，序列号就会对不上，校验依然会失败。

47.TCP和UDP

特性	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
连接性	**面向连接**（必须先三次握手建立连接）	**无连接**（直接发包，不打招呼）
可靠性	**可靠传输**（不丢包、不乱序、不重复）	**不可靠传输**（发出去就不管了，可能丢包）
速度	较慢（因为有复杂的校验和确认机制）	**极快**（头部开销小，没有确认环节）
通信形式	点对点（一对一）	一对一、一对多（广播）、多对多（组播）
数据流	面向字节流（数据没有明确界限）	面向报文（保留数据边界，整发整取）
头部开销	较大（最小 20 字节）	很小（固定 8 字节）

在选择 TCP 还是 UDP 时，核心的考量点在于：**你是要“绝对的准确”，还是要“极致的速度”？**

以下是它们各自最典型、最常见的使用场景：

一、 TCP 的使用场景：追求“稳”

TCP 适用于对**数据完整性**要求极高，且能容忍一定延迟的场景。

1. **网页浏览 (HTTP / HTTPS)**：
    - **原因**：网页由 HTML 代码、CSS 样式和脚本组成。如果丢掉一个包，可能导致页面乱码、样式缺失或功能失效。
2. **文件传输 (FTP / SFTP)**：

- **原因**：下载一个安装包或发送一份文档，必须保证 100% 准确，错一个比特（bit）文件就可能损坏无法打开。

3. **邮件收发 (SMTP / POP3 / IMAP)**：
   - **原因**：邮件内容不能丢字落句，附件也必须完整。
4. **远程登录 (SSH / Telnet)**：
   - **原因**：在命令行操作服务器，指令必须精准到达，且需要确认执行结果。
5. **数据库操作**：
   - **原因**：对数据库的增删改查需要强一致性，不能接受数据丢失或乱序。

二、 UDP 的使用场景：追求“快”

UDP 适用于对**实时性**要求极高，且能容忍少量数据丢失（丢包）的场景。

1. **实时音视频通话 (VoIP)**：
   - **例子**：微信视频、Zoom、Skype。
   - **原因**：通话时，如果某个瞬间网络抖动，画面卡一下（丢包）没关系，重要的是不能因为 TCP 重传导致对话出现几秒钟的延迟。
2. **在线实时游戏**：
   - **例子**：王者荣耀、英雄联盟、FPS 游戏。
   - **原因**：玩家的位置和动作必须实时同步。如果使用 TCP，一旦丢包触发重传，游戏就会出现“瞬移”或卡顿，严重影响体验。
3. **视频直播**：
   - **原因**：直播观众希望看到实时的画面。虽然现在的直播也有几秒延迟，但底层协议为了高并发和低延迟，大量采用了基于 UDP 的方案（如 HTTP-FLV 或 WebRTC）。
4. **域名解析 (DNS)**：
   - **原因**：DNS 查询通常只有一个请求包和一个返回包。用 TCP 握手太慢了，用 UDP 瞬间完成，如果失败了直接重新查一次就行。
5. **物联网设备 (IoT)**：
   - **原因**：很多传感器设备电量有限、性能弱，UDP 的头部开销极小（只有 8 字节），比 TCP（20 字节起）更省电、更省带宽。

---

1. 超时重传（Retransmission）—— 丢包了怎么办?

这是 TCP 保证可靠性的底线。

- **工作原理**：发送方每发一个包，都会启动一个定时器。如果在规定时间内没有收到对方的 **ACK（确认应答）**，就认为包丢了，于是重新发送。
- **重传时间（RTO）怎么定?**
  - RTO（超时重传时间）应该略大于 **RTT（往返时间，即包开一个来回的时间）**。
  - 网络是动态的，所以 RTO 会根据当前的 RTT **动态计算**。如果网络很卡，RTO 会变长。
- **快速重传（Fast Retransmission）**：
  - 如果发送方连续收到 3 个相同的 ACK（比如都在要 5 号包），说明 5 号包肯定丢了。此时不等定时器到期，**立即重传** 5 号包，提高效率。

---

2. 流量控制（Flow Control）—— 对方处理得过来吗?

这是**点对点**的控制，目的是防止发送方发得太快，把接收方的**缓存区（Buffer）**撑爆。

- **核心工具：滑动窗口（Sliding Window）**。

- **工作过程**：
  1. 接收方在返回 ACK 时，会带上一个 `Window Size`（窗口大小），告诉发送方：“我现在的缓存区还有多少空间”。
  2. 发送方根据这个数值来调整自己的发送速度。
  3. 如果接收方告诉发送方 `Window = 0`，发送方就会停止发送，直到收到窗口更新的通知。
- **目的**：避免“收件人仓库满了，快递员还在拼命往门口堆货”的情况。

---

3. 拥塞控制（Congestion Control）—— 网络路况堵吗？

流量控制看的是“对方”，而拥塞控制看的是“**整个互联网的路况**”。如果网络已经很堵了，大家还拼命发包，整个网络就会瘫痪（死锁）。

TCP 采用了四种算法协同工作：

1. **慢启动（Slow Start）**：
   - 刚开始连接时，先发 1 个包，收到确认后翻倍发 2 个，再收到发 4 个、8 个... 呈 **指数级增长。**
   - 目的是快速探测网络承受上限。
2. **拥塞避免（Congestion Avoidance）**：
   - 当包的数量达到一个阈值（ssthresh）后，不再翻倍，而是**线性增加**（每次只加 1 个），稳扎稳打。
3. **拥塞发生（快速重传/超时）**：
   - 一旦出现丢包，认为网络堵了。
   - 此时立即把发送量减半（或者直接降到 1），并将阈值调低。
4. **快速恢复（Fast Recovery）**：
   - 如果是因为收到 3 个重复 ACK 触发的重传（说明网络还没堵死），则直接进入线性增长阶段，而不是从 1 开始慢启动。

总结对照表

机制	解决的问题	关注的对象	核心手段
超时重传	数据丢失	数据的完整性	定时器、ACK、快速重传
流量控制	接收方处理不赢	接收方的缓存空间	**滑动窗口 (RWND)**
拥塞控制	网络拥堵/瘫痪	整个网络的承载力	**拥塞窗口 (CWND)**、慢启动、拥塞避免

48.http1.1 http2 http3区别

HTTP 协议的演进史，本质上是一部**不断压榨网络带宽、降低通信延迟**的斗争史。

一、 HTTP/1.1：持久连接与队头阻塞

1997年发布，是目前互联网上应用最广泛的版本。

- **核心改进：持久连接 (Keep-Alive)。**允许在同一个 TCP 连接上发送多个请求，避免了每次请求都要三次握手的开销。
- **管道化 (Pipelining)**：理论上可以不等响应就发下一个请求，但由于兼容性问题，实际并未普及。
- **致命缺陷：队头阻塞 (Head-of-Line Blocking)。**在一个 TCP 连接中，请求必须按顺序处理。如果第一个请求卡住了（比如服务器处理慢），后面的请求即便再小也得排队。

二、 HTTP/2：二进制分帧与多路复用

2015年发布，基于 Google 的 SPDY 协议。它在不改变 HTTP 语义（方法、状态码等）的前提下，彻底重构了传输方式。

- **二进制分帧 (Binary Framing)**：将信息由文本改为二进制格式，解析更高效。
- **多路复用 (Multiplexing)**：**这是最大的飞跃。**它允许在同一个连接上并发发送无数个请求和响应，不再需要按顺序排队。解决了 HTTP 层的队头阻塞。
- **头部压缩 (HPACK)**：通过静态/动态字典，压缩重复的 Header（如 User-Agent），极大地减少了冗余带宽。
- **服务器推送 (Server Push)**：服务器可以预测客户端需要的资源（如 CSS/JS），在客户端请求 HTML 时主动推送过去。

三、 HTTP/3：基于 UDP 的 QUIC

2022年正式发布。虽然 HTTP/2 解决了 HTTP 层的阻塞，但 **TCP 协议本身**也有队头阻塞（如果一个 TCP 包丢了，整个连接都会暂停等待重传）。为了解决这个问题，HTTP/3 直接抛弃了 TCP。

- **基于 QUIC 协议**：QUIC 运行在 **UDP** 之上，但在应用层实现了可靠传输。
- **彻底解决队头阻塞**：在 HTTP/3 中，丢包只会影响那一个受损的流（Stream），其他数据流照常传输，互不干扰。
- **0-RTT 建连**：结合了 TLS 1.3，第一次握手就能带上数据。由于不需要 TCP 的三次握手，连接建立速度极快。
- **连接迁移**：比如你从 Wi-Fi 切换到 5G，TCP 连接会断开（因为 IP 变了），但 HTTP/3 通过“连接 ID”识别用户，网络环境切换时连接**无缝衔接。**

49.v8内核

V8 是由 Google 开发的开源 **高性能 JavaScript 和 WebAssembly 引擎。**它用 C++ 编写，不仅是 Chrome 浏览器的灵魂，也是 Node.js 能够诞生的核心基础。

V8 的出现彻底改变了 JavaScript 的执行效率，将其从一种“慢速脚本语言”变成了可以处理复杂逻辑的高性能语言。

---

1. V8 的核心执行流程

V8 并不像传统的解释器那样逐行解释执行，它采用了 **JIT (Just-In-Time) 及时编译**技术。

- **解析 (Parser)**：将源代码转化为 **抽象语法树 (AST)。**
- **解释器 (Ignition)**：将 AST 转化为 **字节码 (Bytecode)** 并立即开始执行。此时代码运行速度尚可，但内存占用小。
- **编译器 (TurboFan)**：监控代码运行。如果某段代码（如一个循环）被多次调用，它会被标记为“热点代码”，优化编译器会将其直接编译为机器码（Machine Code），性能直接起飞。
- **反优化 (Deoptimization)**：如果代码的输入类型突然变了（比如之前一直传数字，突然传了个字符串），优化后的机器码会失效，V8 会回退到字节码执行。

---

2. 内存管理与垃圾回收 (GC)

V8 自动管理内存，主要通过**分代回收**机制：

- **新生代 (Young Generation)**：存放存活时间短的对象（如函数局部变量）。使用 Scavenge 算法，清理非常频繁且速度极快。

- **老生代 (Old Generation)**：存放存活时间长或体量巨大的对象。使用 **Mark-Sweep (标记清除)** 和 **Mark-Compact (标记整理)** 算法，防止内存碎片化。
- **Orinoco 优化**：现代 V8 使用并行、并发和增量标记技术，极大地减少了垃圾回收导致的“全停顿（Stop-The-World）”时间，让用户感觉不到卡顿。

---

3. V8 为什么这么快？（核心优化技术）

除了 JIT，V8 还用了两招杀手锏：

A. 隐藏类 (Hidden Classes)

JavaScript 是动态语言，对象属性可以随时增删。V8 为了像 Java 那样快速定位属性内存地址，会在后台为每个对象创建“隐藏类”。

- **建议**：在构造函数中一次性初始化所有属性，且保持属性顺序一致，这样能复用隐藏类，提升性能。

B. 内联缓存 (Inline Caching)

V8 会记住之前查找属性的结果。下次执行相同代码时，直接跳过查找过程，从缓存里拿位置。

49.vite和webpack

---

1. Vite 配置 (`vite.config.ts`)

Vite 的配置非常直观，通常只需要安装对应的插件即可。

核心配置项

- `plugins`：插件数组。Vue 项目必备 `@vitejs/plugin-vue`。
- `resolve.alias`：路径别名。将 `@` 指向 `src`，避免深层嵌套的 `../../../`。
- `server`：开发服务器配置。包括端口、自动打开浏览器、**跨域代理 (proxy)**。
- `build`：打包配置。如输出目录、是否开启压缩、静态资源分包。

实战示例

TypeScript

```
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
import { resolve } from 'path'

export default defineConfig({
  // 插件
  plugins: [vue()],
  // 路径别名
  resolve: {
    alias: {
      '@': resolve(__dirname, 'src') // 配置 @ 别名
    }
  },
  // 开发服务器配置
  server: {
    proxy: {
      '/api': { // 解决“硅谷甄选”项目的跨域问题
        target: 'http://sph-api.atguigu.cn',
        changeOrigin: true,
```

```
        rewrite: (path) => path.replace(/^\/api/, '')
      }
    }
  }
})
```

---

2. Webpack 配置 (`webpack.config.js`)

Webpack 的配置相对复杂，由五个核心概念组成：**Entry, Output, Loader, Plugin, Mode**。

核心配置项

- `entry`: 入口文件。
- `output`: 出口文件及路径。
- `module.rules` **(Loaders)**: 关键！让 Webpack 识别非 JS 文件（如 `.css` 使用 `css-loader`，图片使用 `url-loader`）。
- `plugins`: 执行更广泛的任务。如 `HtmlWebpackPlugin` 自动生成 HTML 并引入 JS。
- `devServer`: 类似 Vite 的 server，提供热更新和代理功能。

实战示例

JavaScript

```
const path = require('path');
const HtmlWebpackPlugin = require('html-webpack-plugin');

module.exports = {
  mode: 'development',
  // 入口文件
  entry: './src/main.js',
  // 出口文件及路径
  output: {
    path: path.resolve(__dirname, 'dist'),
    filename: 'bundle.[contenthash:8].js', // 增加哈希值利用浏览器缓存
    clean: true, // 每次打包前清空 dist
  },
  // 模块
  module: {
    rules: [
      { test: /\.css$/, use: ['style-loader', 'css-loader'] },
      { test: /\.ts$/, use: 'ts-loader' }
    ]
  },
  // 插件
  plugins: [
    new HtmlWebpackPlugin({ template: './public/index.html' })
  ]
};
```

---

3. 两者配置逻辑的本质区别

维度	Vite	Webpack
文件类型支持	**原生支持。**无需配置即可识别 CSS, Less, JSON, TS	**全靠 Loader。**不配 `css-loader` 连 CSS 都打不开
路径别名	简单配置 `alias` 即可	需要配置 `resolve.alias`
环境变量	使用 `.env` 文件，通过 `import.meta.env` 访问	需要使用 `DefinePlugin` 插件手动注入
配置文件	默认支持 TS 配置文件	默认仅支持 JS (需额外安装才能用 TS)

50.浏览器缓存机制

浏览器的缓存机制是前端性能优化的核心手段，其本质是**“用空间换时间”。**通过将请求过的资源（HTML、JS、CSS、图片等）存储在本地，浏览器可以减少冗余的网络请求，显著提升页面加载速度。

浏览器的缓存过程主要分为两个阶段：**强缓存**和**协商缓存。**

---

1. 缓存执行全流程

当浏览器发起一个资源请求时，会遵循以下逻辑：

1. **先找强缓存**：检查本地缓存，如果没过期，直接使用（状态码 **200 OK (from cache)**），**不发请求到服务器。**
2. **再找协商缓存**：如果强缓存失效了，浏览器会发一个携带“缓存标记”的请求去问服务器：“这文件变过吗？”。
    - **没变**：服务器返回 **304 Not Modified**，不传数据体，浏览器继续用本地旧文件。
    - **变了**：服务器返回 **200 OK** 和全新的资源。

---

2. **强缓存 (Strong Cache)**

强缓存完全由客户端控制，不与服务器交互。在 Chrome 的 Network 面板中表现为 `from memory cache` 或 `from disk cache`。

核心字段：

- **Cache-Control (HTTP/1.1 - 推荐)**：使用相对时间。
    - `max-age=3600`：表示资源在 1 小时内有效。
    - `no-cache`：**误导性最强的值。**它指不使用强缓存，直接跳到“协商缓存”阶段。
    - `no-store`：真正意义上的不缓存，也不存储。
    - `public/private`：决定是否允许中间代理（如 CDN）缓存。
- **Expires (HTTP/1.0 - 基本废弃)**：使用绝对时间（服务器日期）。
    - 由于依赖客户端本地时间，如果用户改了系统时钟，缓存就会乱套。

---

3. 协商缓存 (Negotiation Cache)

当强缓存失效，浏览器需要向服务器确认资源是否真正发生变化。

核心字段（成对出现）：

第一对：基于修改时间

- **Last-Modified**（响应头）：服务器告诉浏览器文件最后修改时间。

- **If-Modified-Since**（请求头）：下次请求时，浏览器带上这个时间问服务器。
- **缺点**：精度只有秒级；如果文件内容没变但时间变了（比如重新覆盖了文件），会导致缓存失效。

第二对：基于文件指纹（ETag - 推荐）

- **ETag**（响应头）：服务器根据文件内容生成的唯一哈希值（指纹）。
- **If-None-Match**（请求头）：下次请求带上这个哈希值。
- **优点**：最精准。只要内容动了一个比特，哈希值就会变。

---

4. 缓存存储位置

你在 Network 面板常看到的两种标注：

- **Memory Cache (内存缓存)**：
  - **特点**：读取极快，但进程关闭（关掉标签页）即消失。
  - **存什么**：通常是小脚本、图片、Base64。
- **Disk Cache (硬盘缓存)**：
  - **特点**：读取稍慢，但持久存储，重启电脑依然存在。
  - **存什么**：大文件、CSS、JS。
- **Service Worker Cache**：
  - 开发者可以手动控制哪些文件被缓存，常用于离线应用（PWA）。

一、 强缓存 (Strong Cache)

强缓存是效率最高的缓存方式。当浏览器请求某个资源时，会先查看该资源是否命中强缓存。如果命中，浏览器**直接读取本地资源**，不会向服务器发送任何请求。

1. 核心特征

- **状态码**：在网络面板中显示为 `200 OK (from memory cache)` 或 `200 OK (from disk cache)`。
- **速度**：由于不经过网络传输，速度极快（通常小于 10ms）。

2. 控制字段（HTTP Header）

强缓存主要由两个响应头字段控制：

- **Cache-Control (HTTP/1.1 - 绝对主流)**：
  - **原理**：使用“相对时间”来控制。
  - **示例**：`Cache-Control: max-age=3600`。
  - **含义**：告诉浏览器，这个资源从现在开始，1小时（3600秒）内都是有效的，别来烦服务器。
  - **常用值**：`public`（所有人都能存）、`private`（只有浏览器能存）、`no-store`（绝对不存）。
- **Expires (HTTP/1.0 - 基本弃用)**：
  - **原理**：使用“绝对时间”。
  - **示例**：`Expires: Wed, 21 Oct 2026 07:28:00 GMT`。
  - **漏洞**：它依赖客户端本地时间。如果你手动把电脑时间调慢一小时，这缓存可能就永远“有效”了。

二、 协商缓存 (Negotiation Cache)

当强缓存失效（过期了），或者你设置了 `Cache-Control: no-cache` 时，浏览器就必须去问服务器了。这个“询问并决定是否使用旧资源”的过程就是**协商缓存**。

1. 核心特征

- **状态码**：
  - 如果服务器说“没变”，返回 **304 Not Modified**（只传头信息，不传内容，省流量）。
  - 如果服务器说“变了”，返回 **200 OK**（传完整的新内容）。
- **速度**：比强缓存慢（因为有往返请求），但比重新下载大文件快得多。

2. 控制字段（成对出现）

A. 比较修改时间（精度较低）

- **Last-Modified** (服务器发)：这文件上次修改是 10:00。
- **If-Modified-Since** (浏览器带)：11:00 时强缓存过期，浏览器问：“这文件 10:00 之后改过吗？”。
- **缺点**：只能精确到秒。如果一秒内改了多次，或者文件没变但修改时间变了，它会误判。

B. 比较文件指纹 (ETag - 推荐)

- **ETag** (服务器发)：这文件的哈希值是 `abc123`。
- **If-None-Match** (浏览器带)：下次请求问：“这文件的哈希值还是 `abc123` 吗？”。
- **优点**：最精准。只要内容动了一个比特，哈希值就会变，完美解决时间戳的问题。

51.浏览器性能优化

浏览器的性能优化是一个系统工程，通常遵循**“减少请求、减小体积、加快渲染、优化体验”**的原则。

我们可以根据网页从输入 URL 到最终显示的生命周期，将优化策略分为以下几个维度：

---

1. 网络传输层优化（加载快）

这是优化的第一步，目标是让资源以最快的速度到达浏览器。

- **利用缓存（核心）**：
  - **强缓存**：对带有哈希值的静态资源（JS/CSS/图片）设置长期的 `Cache-Control: max-age`。
  - **协商缓存**：利用 `ETag` 或 `Last-Modified` 确保资源更新时客户端能及时感知。
- **开启 HTTP/2 或 HTTP/3**：
  - 利用**多路复用**解决 HTTP/1.1 的队头阻塞问题。
  - **头部压缩**减少重复 Header 带来的开销。
- **使用 CDN（内容分发网络）**：
  - 将资源部署在离用户最近的节点，减少网络往返时间（RTT）。
- **开启 Gzip 或 Brotli 压缩**：
  - 在服务器端对文本资源进行压缩，通常能减小 70% 左右的体积。

---

2. **资源体积与构建优化（包体小）**

在你的 **“硅谷甄选”** 等项目中，构建工具（Vite/Webpack）的配置至关重要。

- **Tree Shaking（摇树优化）**：
  - 删除代码中未被引用的无效模块，减小打包体积。
- **分包策略 (Code Splitting)**：
  - 将第三方库（如 Vue, ECharts）与业务代码分离，利用浏览器并行下载。
- **按需加载 (Lazy Loading)**：
  - **路由懒加载**：只有访问对应页面时才加载相关 JS。

    - **图片懒加载**：优先加载可视区域内的图片，剩下的通过 `loading="lazy"` 或监听滚动加载。
- **图片优化**：
    - 使用 WebP 等高压缩比格式；对小图标使用 **SVG Sprite** 或 **Iconfont**。

---

3. 浏览器渲染层优化（运行流利）

这一部分主要是为了减少浏览器的计算压力，避免页面卡顿。

- **减少重排（Reflow）与重绘（Repaint）**：
    - **重排**：改变几何属性（宽、高、位置）。代价极高，会触发整个渲染树计算。
    - **重绘**：改变外观（颜色、背景）。代价相对较小。
    - **优化方案**：使用 `transform` 和 `opacity` 实现动画，这会触发 **GPU 加速**（合成层），不会引发重排重绘。
- **防抖 (Debounce) 与节流 (Throttle)**：
    - 在 **"数据大屏"** 的窗口缩放（resize）或滚动事件中，限制函数执行频率，减轻 CPU 负担。
- **Web Workers**：
    - 将耗时的复杂计算（如大批量数据处理）放在 Web Worker 中运行，避免阻塞主线程（JS 引擎）。

---

4. 关键渲染路径优化（首屏秒开）

- **CSS 放在头部，JS 放在底部**：
    - 防止 JS 阻塞 DOM 解析。
    - 使用 `async`（异步加载，加载完立即执行）或 `defer`（异步加载，DOM 解析完后再执行）。
- **骨架屏 (Skeleton Screen)**：
    - 在数据加载完成前，先展示页面的大致轮廓，缓解用户的焦虑感。
- **预解析与预加载**：
    - `<link rel="dns-prefetch" href="...">`：预解析域名。
    - `<link rel="preload" href="...">`：提前加载关键资源。

52.vue2和vue3的区别

Vue 2 和 Vue 3 的区别不仅仅是版本的迭代，更是底层架构的彻底重构。Vue 3 在性能、体积、开发体验以及对大型项目的支持上都有了质的飞跃。

以下是它们最核心的五个区别：

---

1. 响应式原理的重塑（核心底层）

这是两者最本质的区别，决定了性能和功能的上限。

- **Vue 2：** `Object.defineProperty`
    - **原理**：通过拦截对象属性的 `getter` 和 `setter` 实现。
    - **局限**：无法检测到对象属性的**新增或删除**，无法原生监听**数组下标**的变化。这就是为什么 Vue 2 需要 `this.$set` 的原因。
- **Vue 3：** `Proxy`
    - **原理**：直接代理整个对象，拦截所有操作。
    - **优势**：原生支持对象增删、数组下标修改。且**惰性监听**（只有访问到深层属性时才代理），大大提升了初始化渲染速度。

---

2. 开发模式：Options API vs Composition API

- **Vue 2：Options API（选项式 API）**
  - 代码分散在 `data`，`methods`，`computed`，`watch` 中。
  - **痛点**：当组件逻辑变得复杂时，处理同一个功能的代码会横跨数百行，上下反复横跳，维护极其痛苦。
- **Vue 3：Composition API（组合式 API）**
  - 使用 `setup` 函数，将相同功能的逻辑（数据、方法、生命周期）写在一起。
  - **优势**：逻辑复用极其方便（通过 Hook 函数），代码结构更清晰，对 **TypeScript** 的支持近乎完美。

---

3. 生命周期钩子的更名

Vue 3 的生命周期钩子在组合式 API 中需要先导入，并且去掉了 `beforeCreate` 和 `created`（因为 `setup` 本身就承载了这两个钩子的职责）。

阶段	Vue 2 (Options)	Vue 3 (Composition)
挂载	`beforeMount` / `mounted`	`onBeforeMount` / `onMounted`
更新	`beforeUpdate` / `updated`	`onBeforeUpdate` / `onUpdated`
销毁	`beforeDestroy` / `destroyed`	`onBeforeUnmount` / `onUnmounted`

---

4. 性能优化与体积

- **Diff 算法优化**：Vue 3 引入了 **静态标记（PatchFlag）**。它能识别出哪些 DOM 是静态的（永远不变），哪些是动态的。在更新时，它只对比动态节点，跳过静态节点，效率提升数倍。
- **Hoisting（静态提升）**：Vue 3 编译器会将静态节点的创建提升到渲染函数之外，避免每次渲染都重新创建对象。
- **Tree-shaking（按需编译）**：Vue 3 的全局 API 都是通过 ES 模块导入的。如果你没用到 `keep-alive` 或 `v-model`，打包工具会直接把这部分代码删掉，让最终的包体积更小。

---

5. 碎片化（Fragments）支持

- **Vue 2**：组件必须有且仅有一个根标签（通常外面要包一层无意义的 `<div>`）。
- **Vue 3**：组件可以有**多个根节点**。这减少了 DOM 层级，也让 HTML 结构更简洁。

Code snippet

```
<template>
  <header>...</header>
  <main>...</main>
  <footer>...</footer>
</template>
```